Clustering of Credit Card Customer Data

Training and Testing using KNIME Platform

Overview

This document presents the complete workflow for clustering credit card customer data using three unsupervised machine learning algorithms: K-Means, Hierarchical Clustering, and Density-Based Spatial Clustering of Applications with Noise (DBSCAN). The objective is to document the implementation process and the configuration of each algorithm within KNIME.

The process begins with loading the Credit Card dataset and performing the necessary data preprocessing to prepare it for clustering. The prepared dataset is then used as the input for the individual clustering workflows. Each algorithm is implemented in a separate KNIME workflow file, enabling its configuration, execution, and resulting cluster assignments to be examined independently.

For each workflow, the relevant nodes, their configurations, and their outputs are documented. Explanations are provided for important settings and processing steps to clarify their purpose and contribution to the clustering process. Where applicable, cluster visualizations are included to illustrate the resulting groupings and support the interpretation of the outputs. This document serves as a technical reference for understanding how the clustering algorithms were implemented and executed in KNIME, complementing the accompanying analysis and results report, which compares the algorithms based on clustering quality, visual interpretability, and computational efficiency and explains the rationale for selecting the most suitable approach.

Table of Contents

1. Preparing the Dataset 3
    Importing Data to the Workflow 3
2. Data Preprocessing 3
    2.1 Detect and handle missing values 3
    2.2 Detect and handle outliers 4
    Reduction in Dataset Size 5
    2.3 Data Transformation - Normalization 6
    2.4 Feature Selection 7
3. Model Training and Testing 12
    (a) K-Means Clustering 12
        Dimensionality Reduction 15
        Clustering when k=4 18
        Clustering when k=3 19
        Clustering when k=5 20
        Summary of K-Means Clustering 21
    (b) Hierarchical Clustering 22
    (c) DBSCAN 29

1. Preparing the Dataset

Importing Data to the Workflow

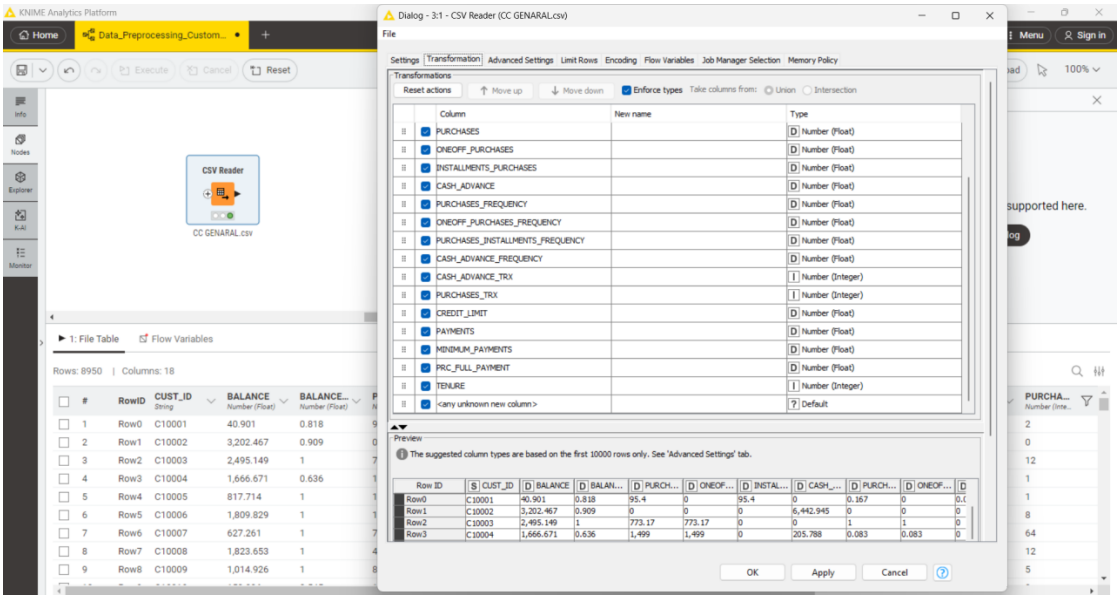


Figure 1: CSV Reader Node Configuration

2. Data Preprocessing

2.1 Detect and handle missing values

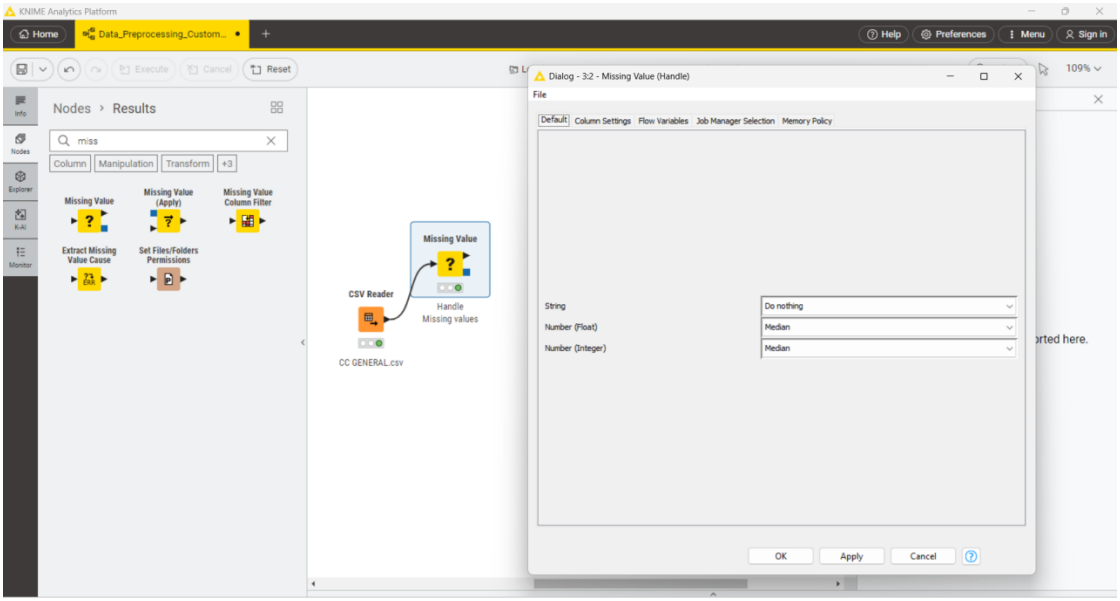


Figure 2: Missing Values Node and Configuration

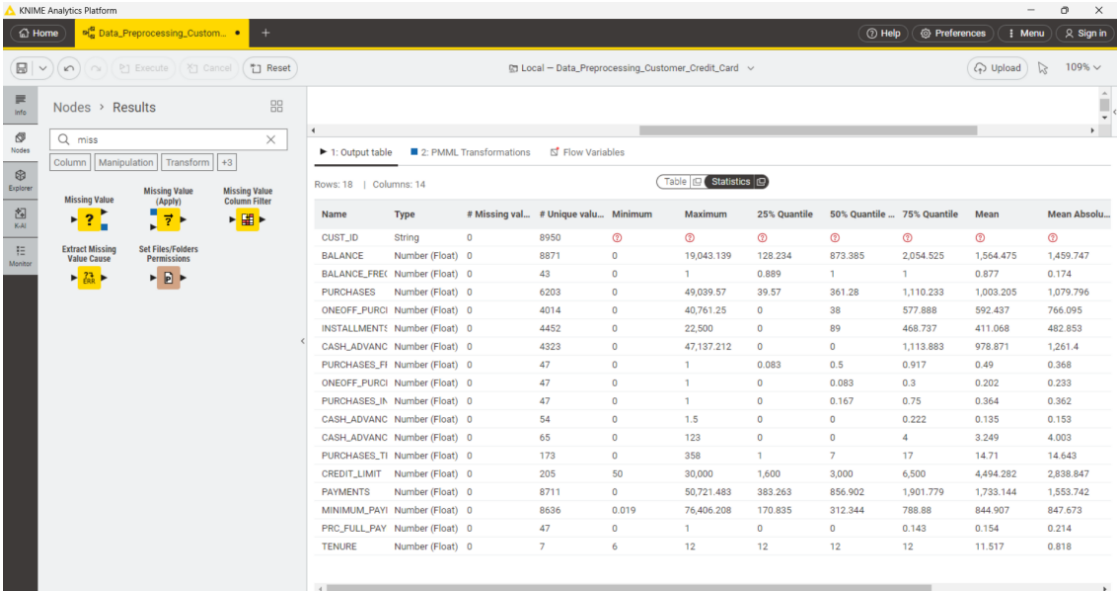


Figure 3: Output of Missing Values Node

2.2 Detect and handle outliers

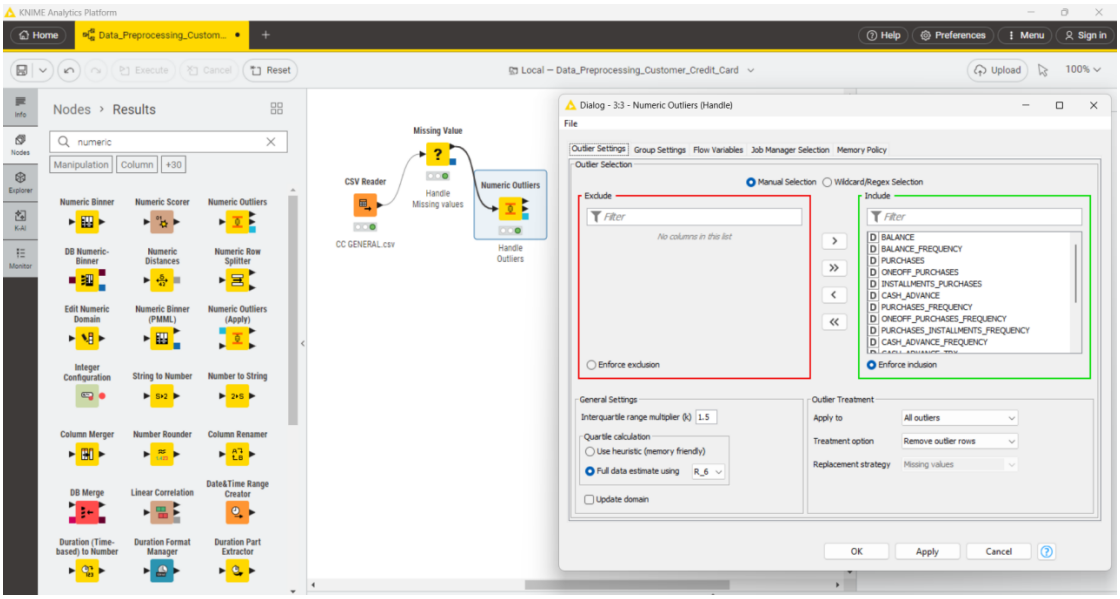
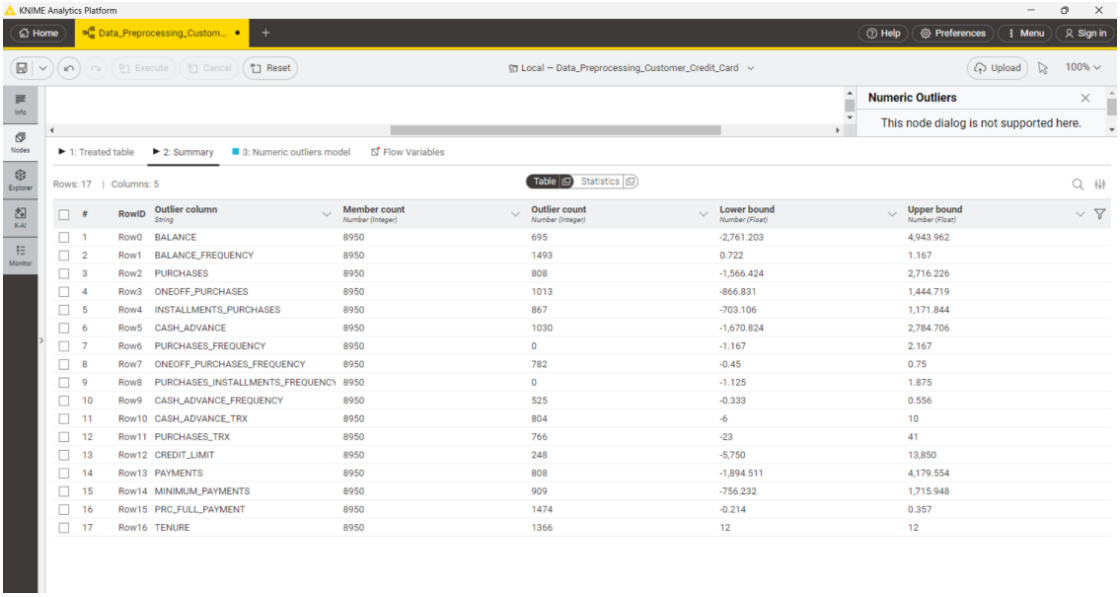


Figure 4: Numeric Outliers Node and Configuration



#	RowID	Outlier column (String)	Member count (Number (Integer))	Outlier count (Number (Integer))	Lower bound (Number (Float))	Upper bound (Number (Float))
1	Row0	BALANCE	8950	695	-2,761.203	4,943.962
2	Row1	BALANCE_FREQUENCY	8950	1493	0.722	1.167
3	Row2	PURCHASES	8950	808	-1,566.424	2,716.226
4	Row3	ONEOFF_PURCHASES	8950	1013	-866.831	1,444.719
5	Row4	INSTALLMENTS_PURCHASES	8950	867	-703.106	1,171.844
6	Row5	CASH_ADVANCE	8950	1030	-1,670.824	2,784.706
7	Row6	PURCHASES_FREQUENCY	8950	0	-1.167	2.167
8	Row7	ONEOFF_PURCHASES_FREQUENCY	8950	782	-0.45	0.75
9	Row8	PURCHASES_INSTALLMENTS_FREQUENCY	8950	0	-1.125	1.875
10	Row9	CASH_ADVANCE_FREQUENCY	8950	525	-0.333	0.556
11	Row10	CASH_ADVANCE_TRX	8950	804	-6	10
12	Row11	PURCHASES_TRX	8950	766	-23	41
13	Row12	CREDIT_LIMIT	8950	248	-5,750	13,850
14	Row13	PAYMENTS	8950	808	-1,894.511	4,179.554
15	Row14	MINIMUM_PAYMENTS	8950	909	-756.232	1,715.948
16	Row15	PRC_FULL_PAYMENT	8950	1474	-0.214	0.357
17	Row16	TENURE	8950	1366	12	12

Figure 5: Statistics after Numeric Outliers Node

Reduction in Dataset Size

When outliers were removed, the dataset size decreased to 2,986 rows. This reduction is expected because the original credit card dataset contains many extreme financial values that fall well outside the typical range of customer behavior.

These values act as statistical anomalies and can interfere with distance-based clustering by pulling centroids toward extreme points and reducing overall cluster quality.

By retaining only consistent, non-extreme records, the dataset becomes more stable and suitable for analysis, allowing the clustering algorithms to form clearer, more meaningful, and better-separated groups.

2.3 Data Transformation - Normalization

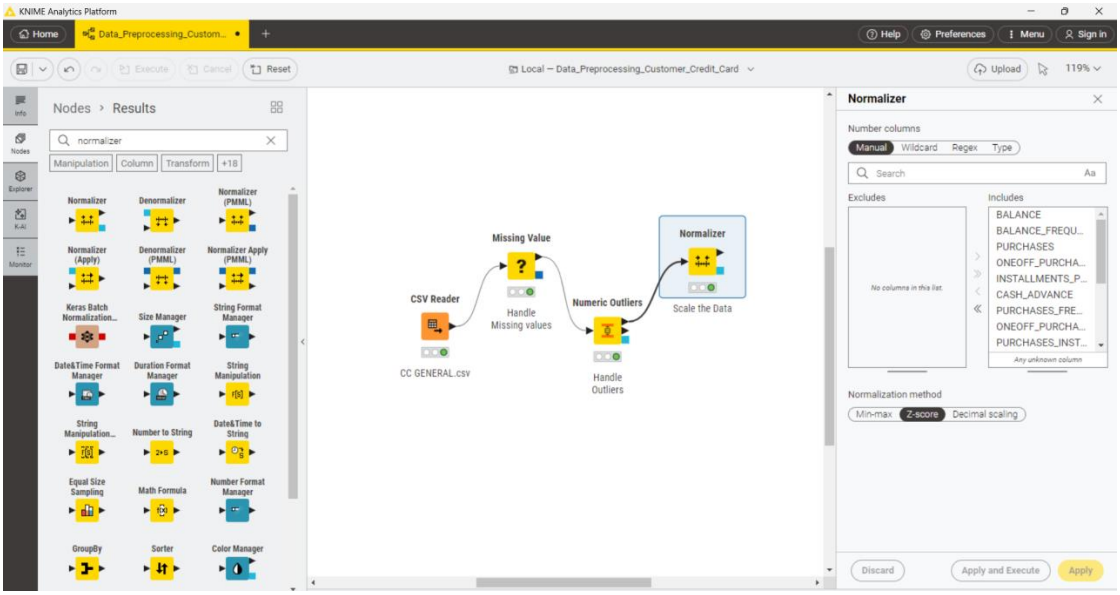


Figure 6: Normalizer Node and Configuration

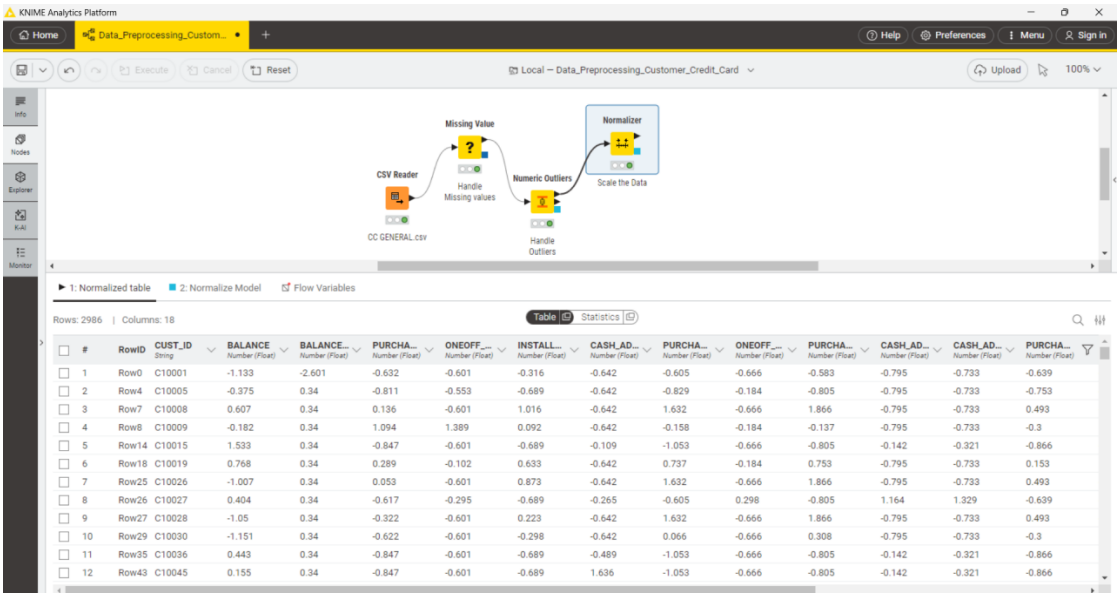


Figure 7: Output Table after Normalization

The dataset was scaled using the **Normalizer node** in KNIME. **Z-Score normalization** was applied because clustering algorithms such as K-Means and Hierarchical Clustering rely on distance calculations. Z-Score standardization transforms all numerical features to have mean = 0 and standard deviation = 1, ensuring that variables with large ranges (e.g., CREDIT_LIMIT) do not dominate smaller ranged variables (e.g., TENURE).

Additional manual configurations were tested for comparison, confirming that Z-Score provided the most stable distribution for distance-based clustering.

2.4 Feature Selection

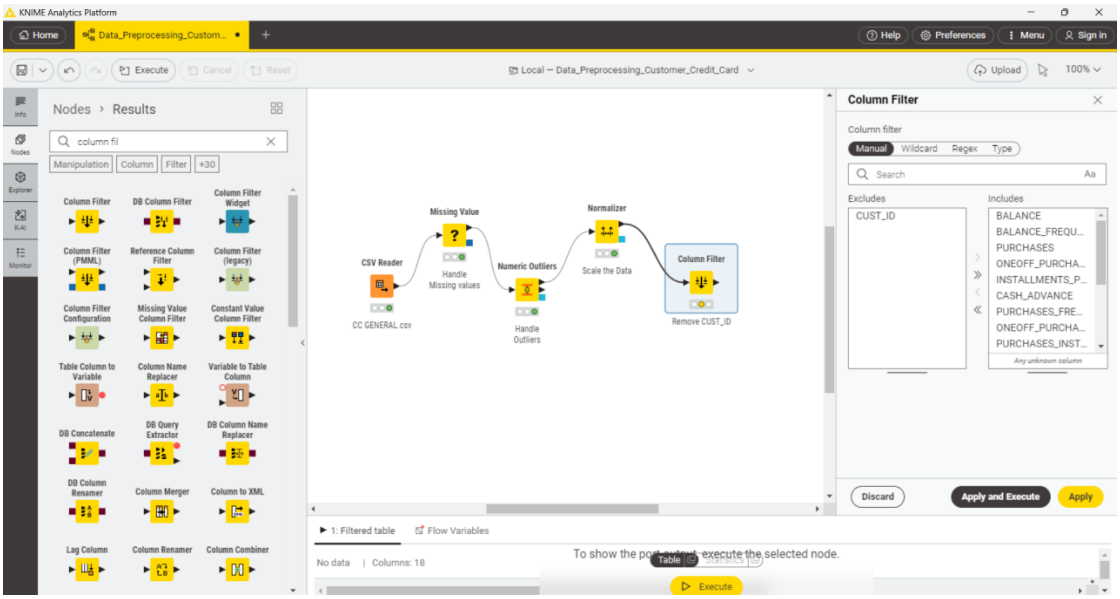


Figure 8: Column Filter Node and Configuration

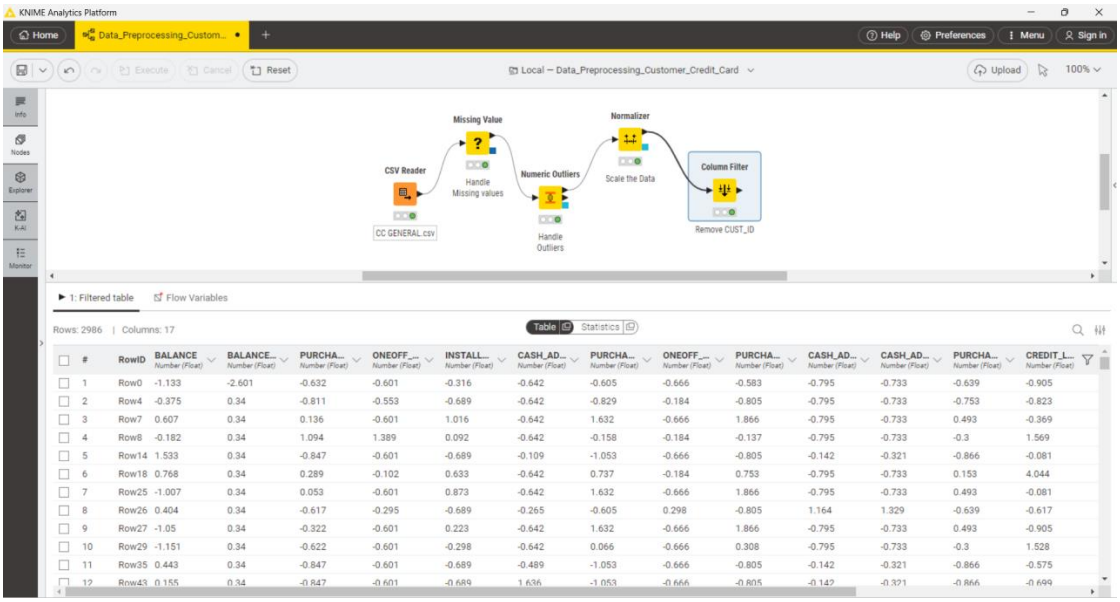


Figure 9: Output Table after Column Filter Node

A **Column Filter node** was used to restrict the analysis to continuous numerical features only.

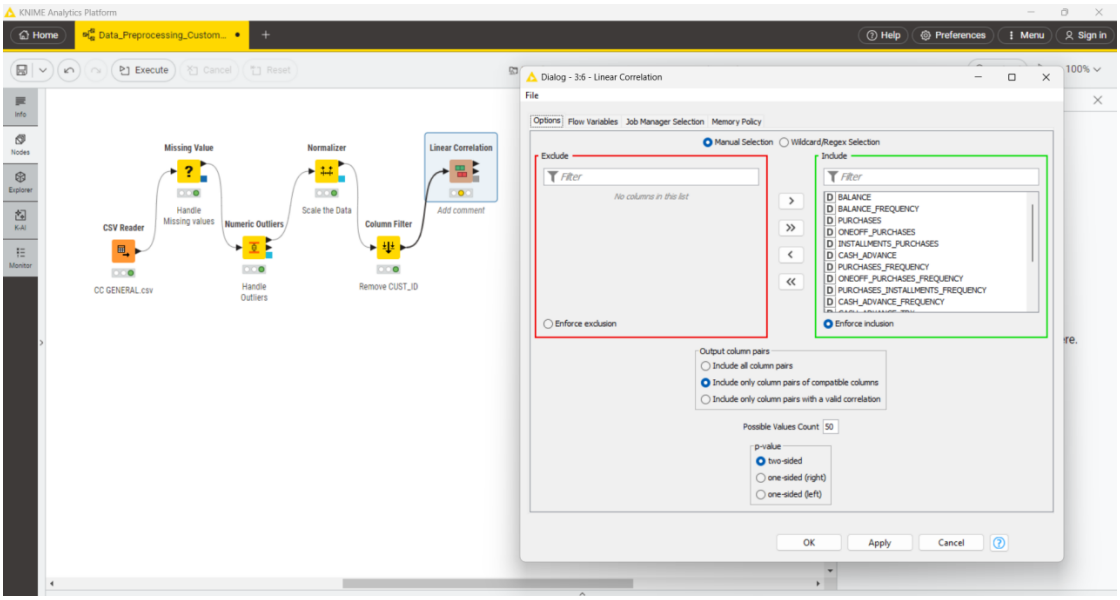


Figure 10: Linear Correlation Node and Configuration

The **Linear Correlation node** was used to identify highly correlated numerical features. **Two-sided p-values** were applied to test both positive and negative correlations, while the **possible-values threshold** ensured only meaningful numerical columns were considered. This helped detect redundant attributes that could bias distance-based clustering.

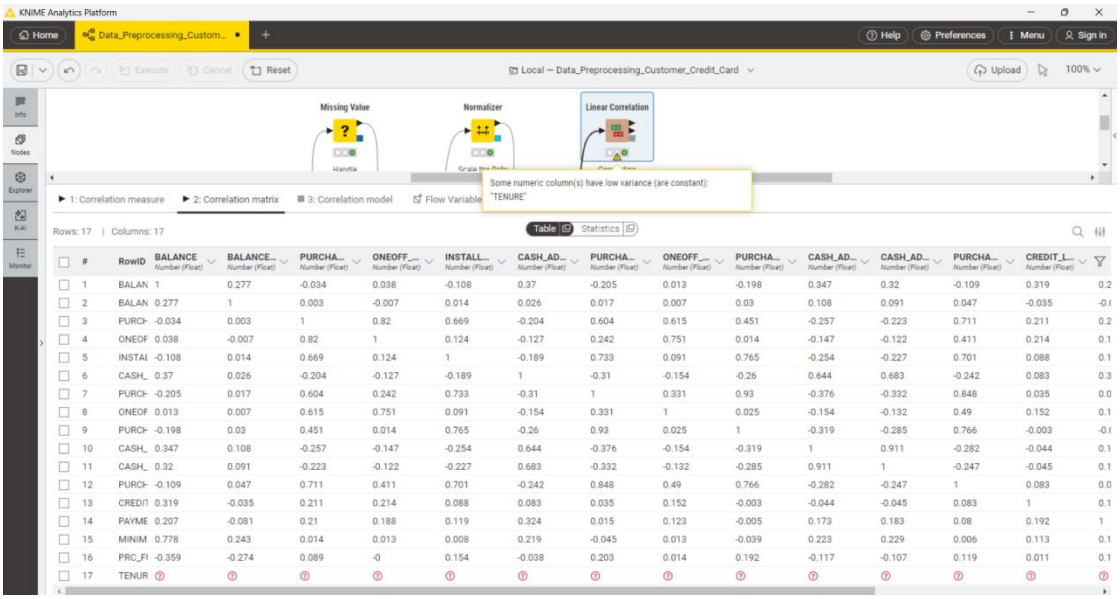


Figure 11: Output Table of Linear Correlation Node - Correlation Metrix

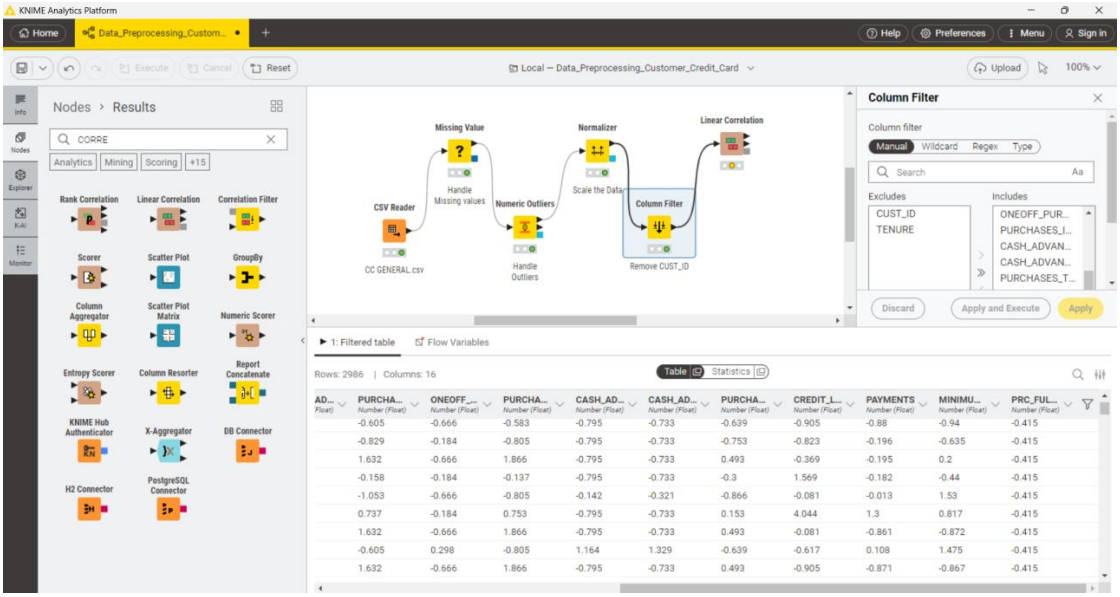


Figure 12: Column Filter Node, Configuration and Output Table (Exclude TENURE together with CUS_ID)

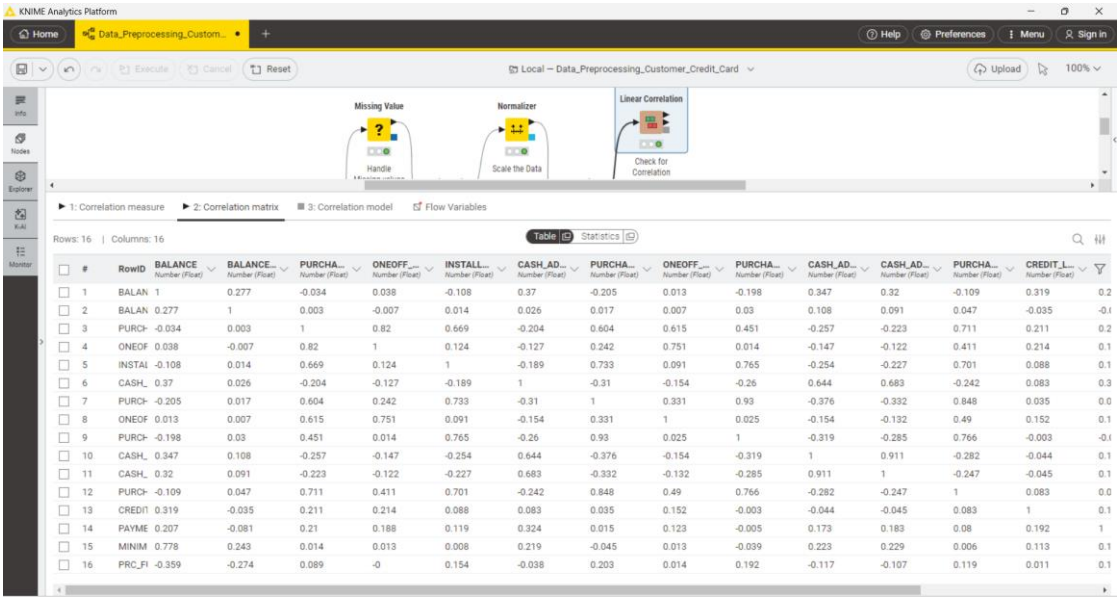


Figure 13: Linear Correlation Filter Node, and Output Table (After Excluding TENURE)

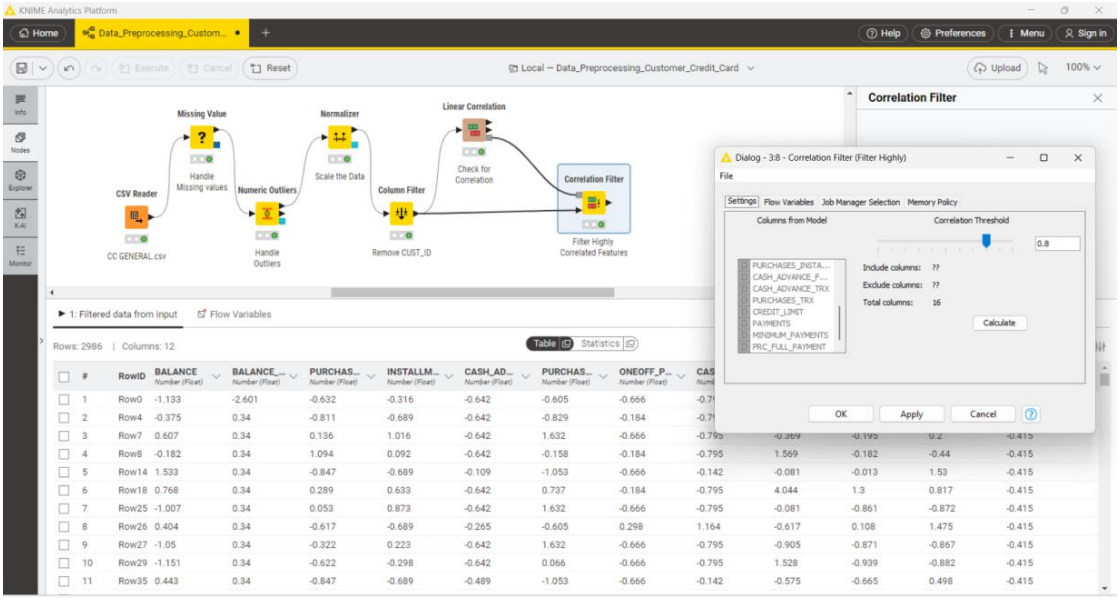


Figure 14: Correlation Filter Node, Configuration and Output Table

The **Correlation Filter** node was configured with a correlation threshold of 0.8 to automatically remove redundant features identified in the **Linear Correlation** model *(4 columns in total were removed)*. This ensured that only independent attributes were retained for model training.

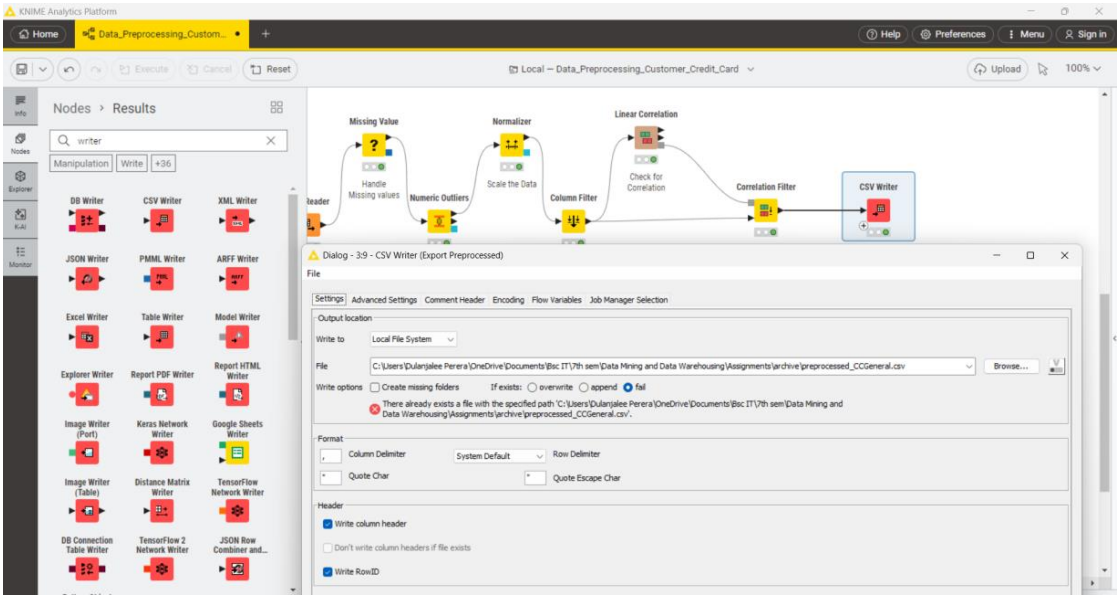


Figure 15: CSV Writer and Configuration

After completing all preprocessing steps—including handling missing values, removing outliers, normalizing the numerical features, and filtering correlated attributes—the final cleaned dataset was exported using a **CSV Writer node**. This

exported file represents a fully standardized and optimized version of the data, suitable for distance-based clustering methods. To maintain workflow clarity and ensure that each clustering algorithm received the exact same preprocessed input, separate KNIME workflows were created for K-Means, Hierarchical Clustering, and DBSCAN. This approach avoids repeating preprocessing steps, keeps each workflow focused on a single model, and makes the evaluation and comparison of clustering results more consistent and reliable.

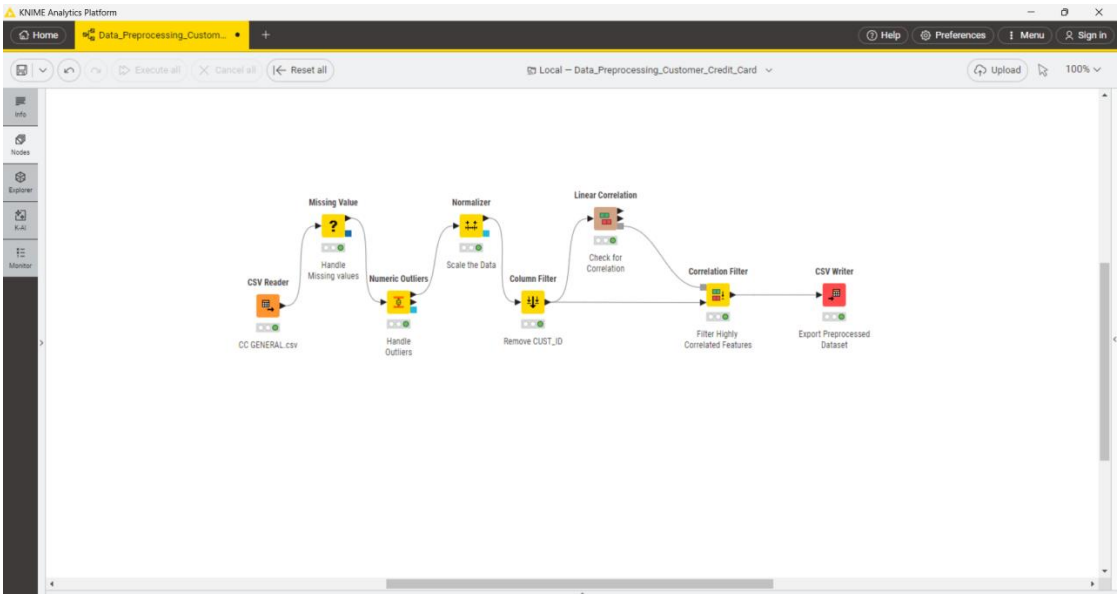


Figure 16: Complete Preprocessing Workflow

3. Model Training and Testing

(a) K-Means Clustering

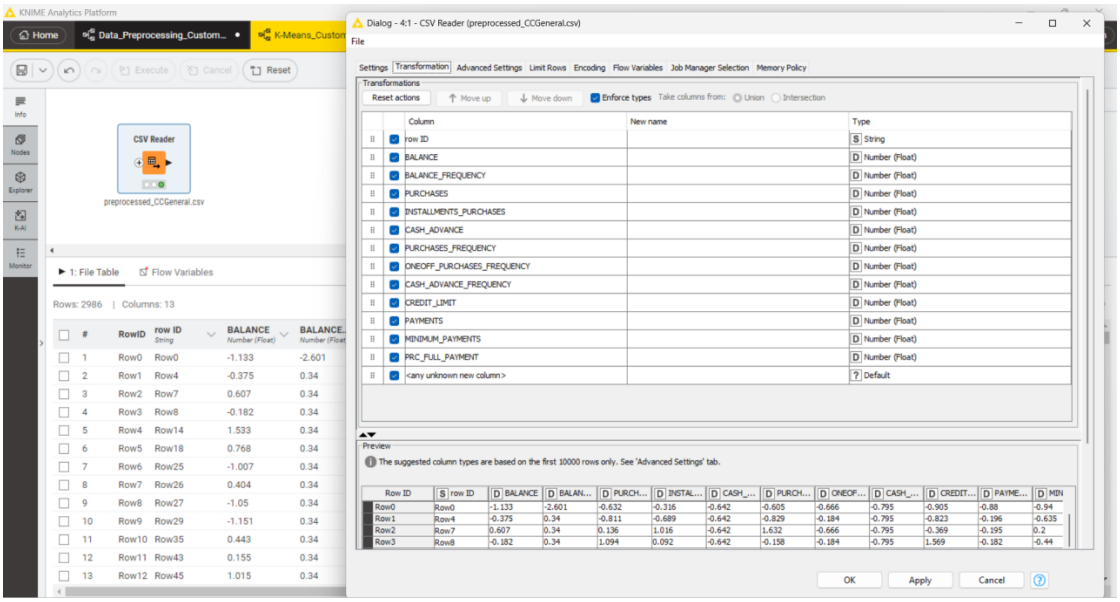


Figure 17: CSV Reader Node, Configuration, and Output Table

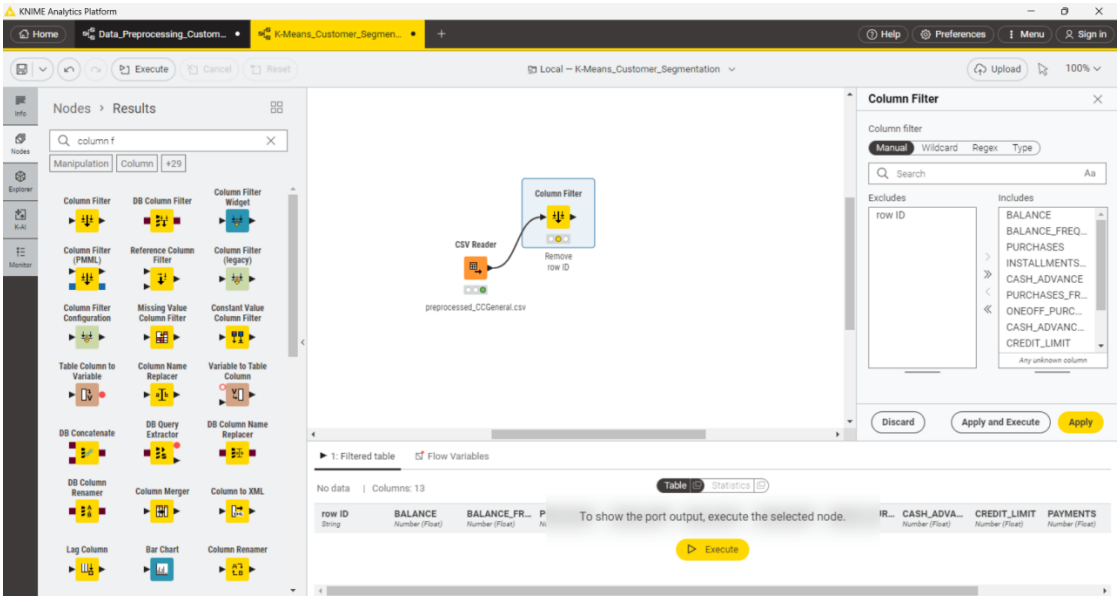


Figure 18: Column Filter Node and Configuration

The dataset contained an identifier column, *row ID*, which does not carry any behavioral or numerical meaning relevant to clustering. Since clustering algorithms rely on distance calculations between feature values, including an ID attribute would introduce noise without contributing to customer segmentation. Therefore, the identifier column was removed using a **Column Filter node** to ensure that only meaningful numerical features were passed into the clustering models.

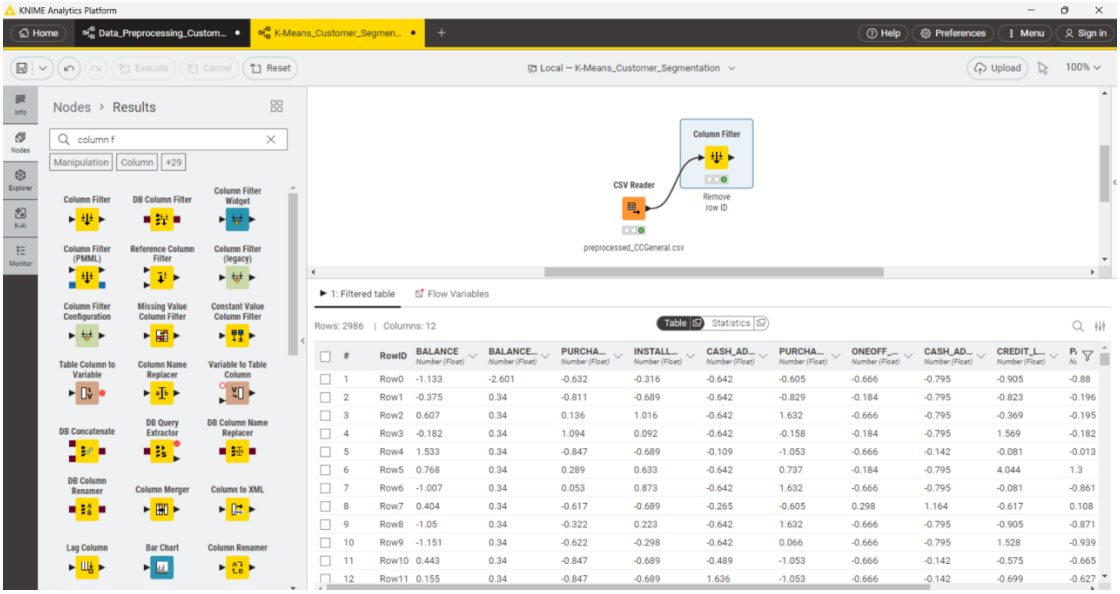


Figure 19: Output Table of Column Filter Node

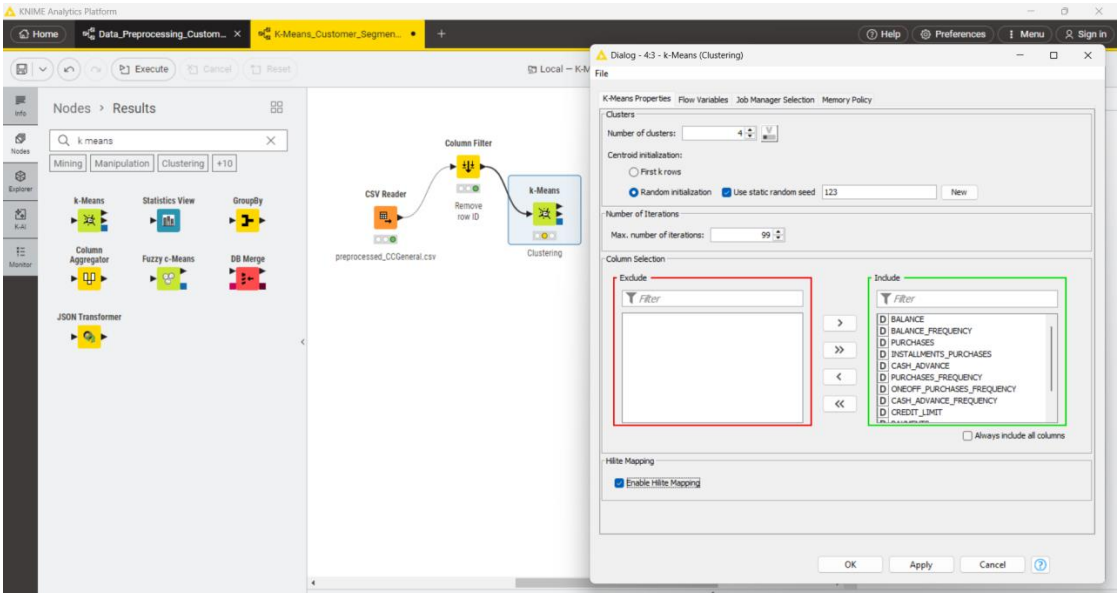


Figure 20: K-Means Node and Configuration

K-Means clustering was implemented using the **K-Means node**. The number of clusters was initially set to four (4), allowing subsequent evaluation using **silhouette scores** to validate the choice of k.

Random initialization was used instead of selecting the first K rows to avoid bias from the dataset's ordering, and a **fixed random seed** (123) was applied to ensure reproducibility.

The default maximum of **99 iterations** was retained, and **hilite mapping** was enabled to facilitate interactive visual exploration of cluster assignments.

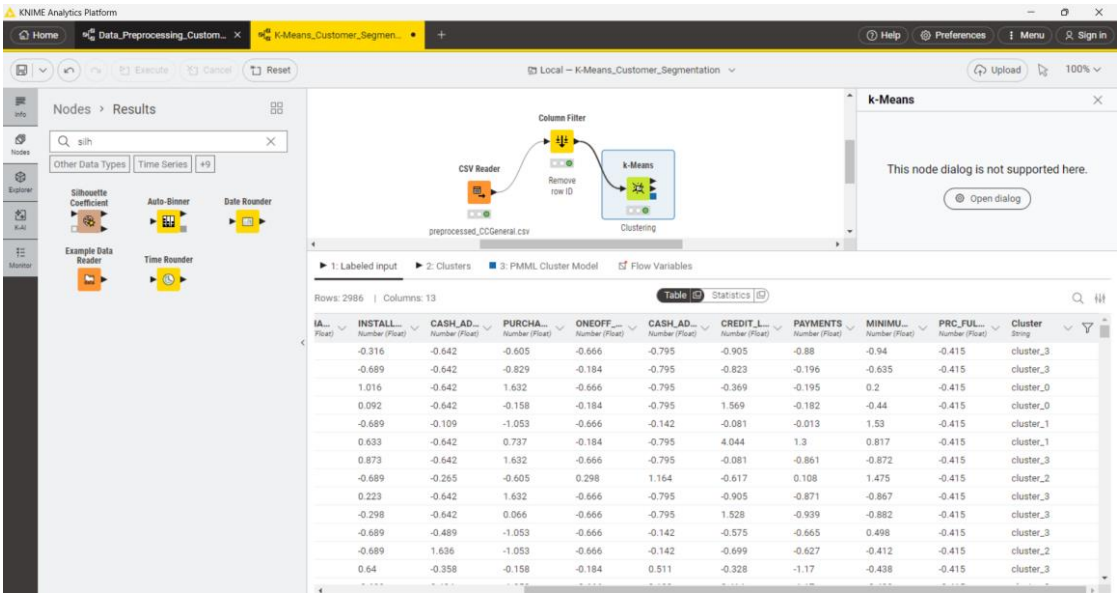


Figure 21: Output Table of K-Means Node

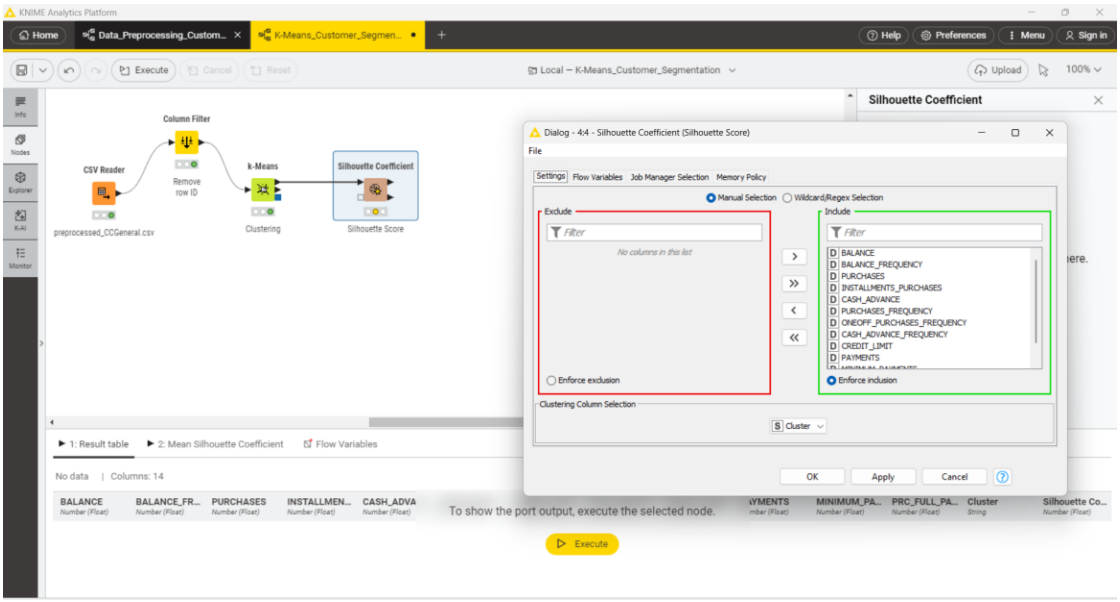


Figure 22: Silhouette Coefficient Node and Configuration

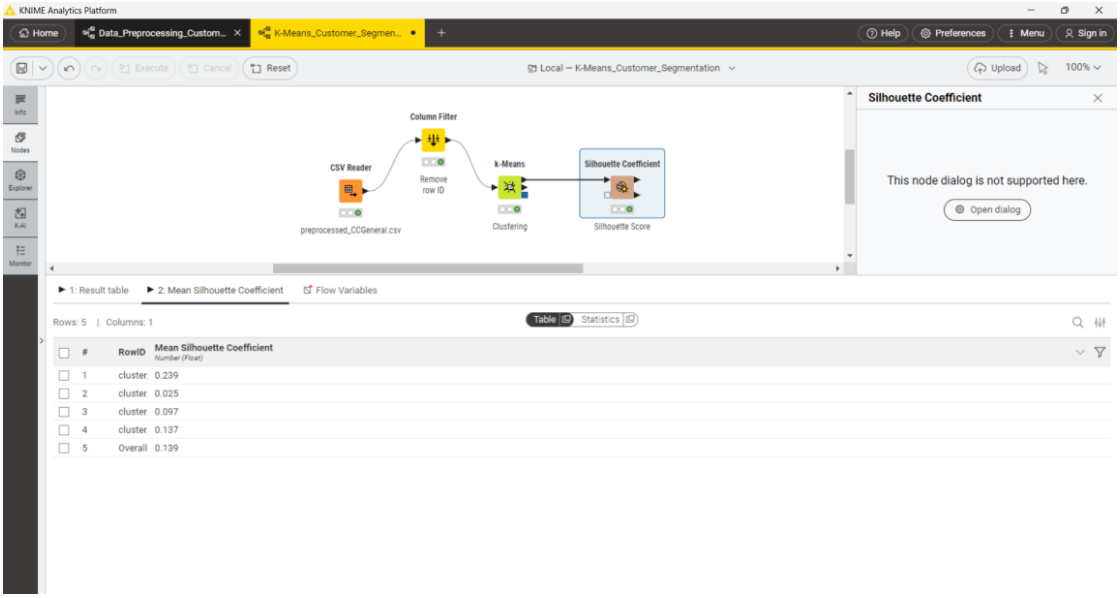


Figure 23: Output of Silhouette Coefficient Node - Mean Silhouette Coefficient

Dimensionality Reduction

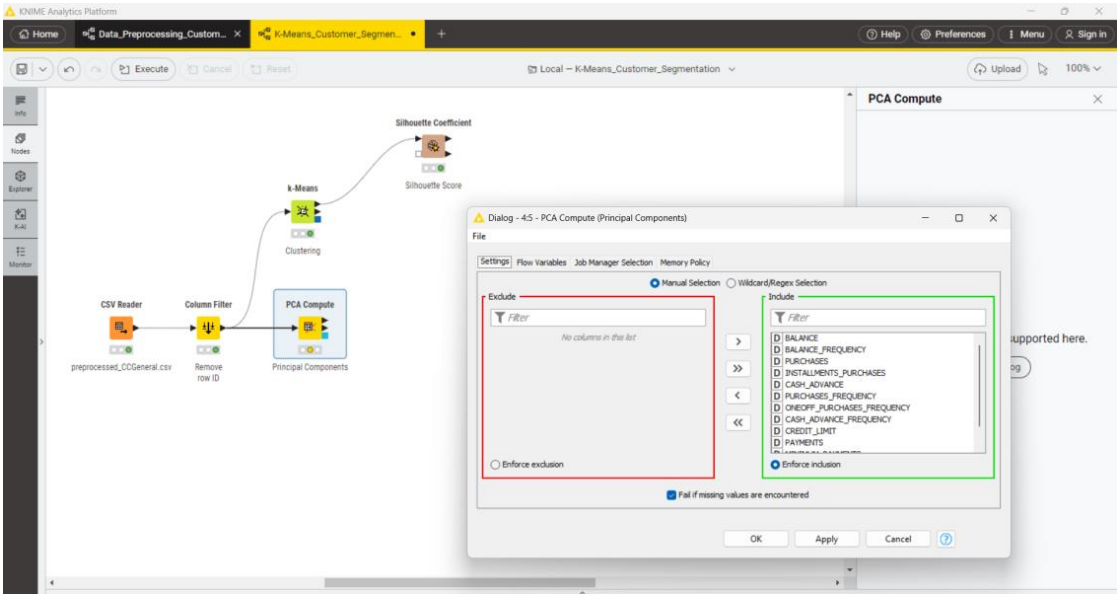


Figure 24: PCA Compute Node and Configuration

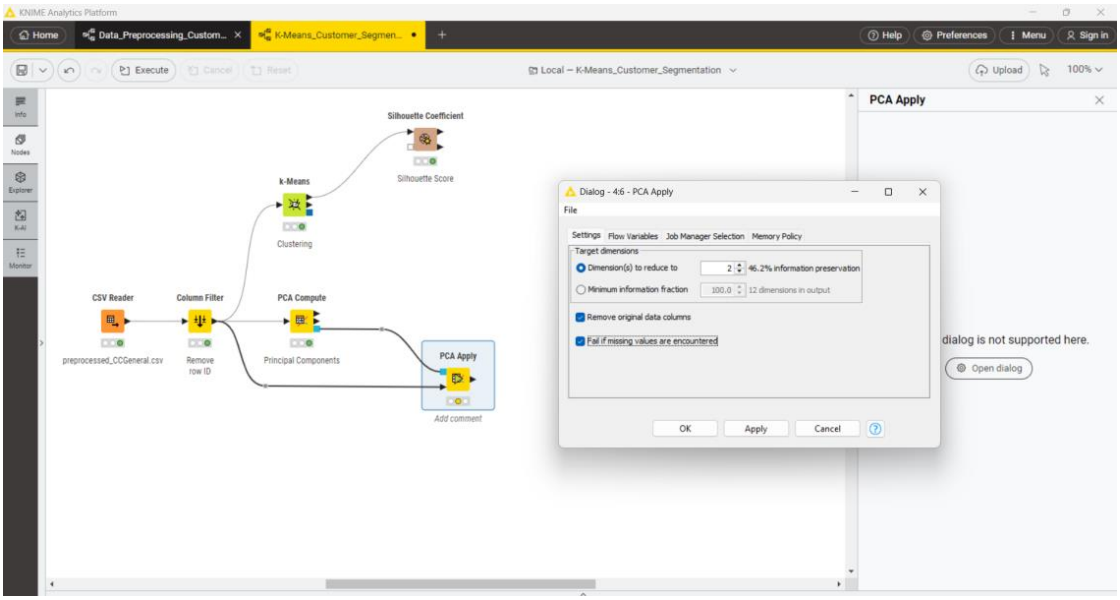


Figure 25: PCA Apply Node and Configuration

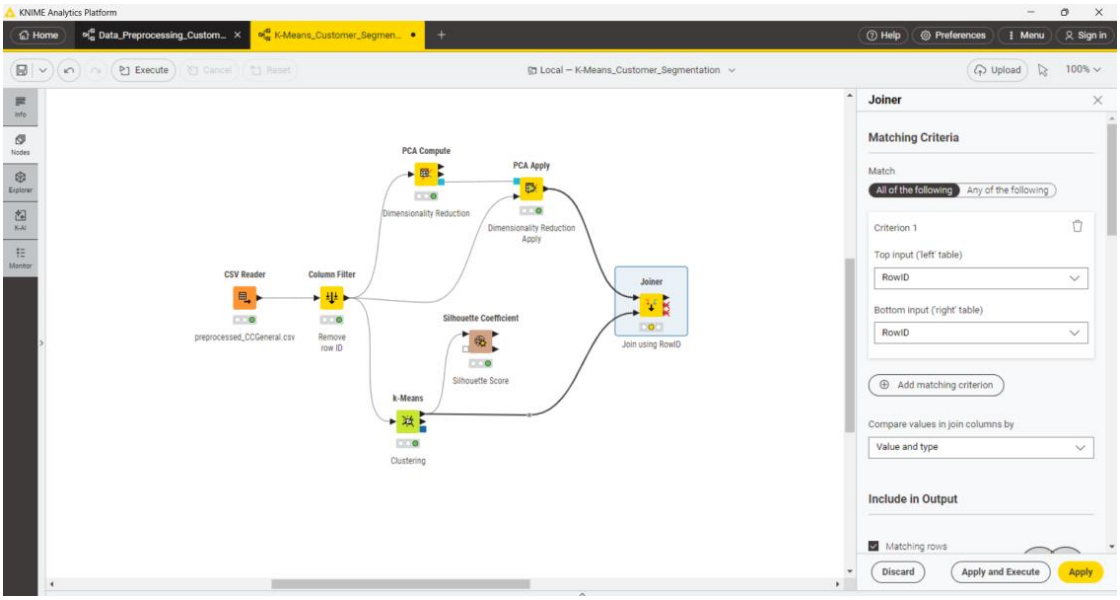


Figure 26: Joiner Node and Configuration

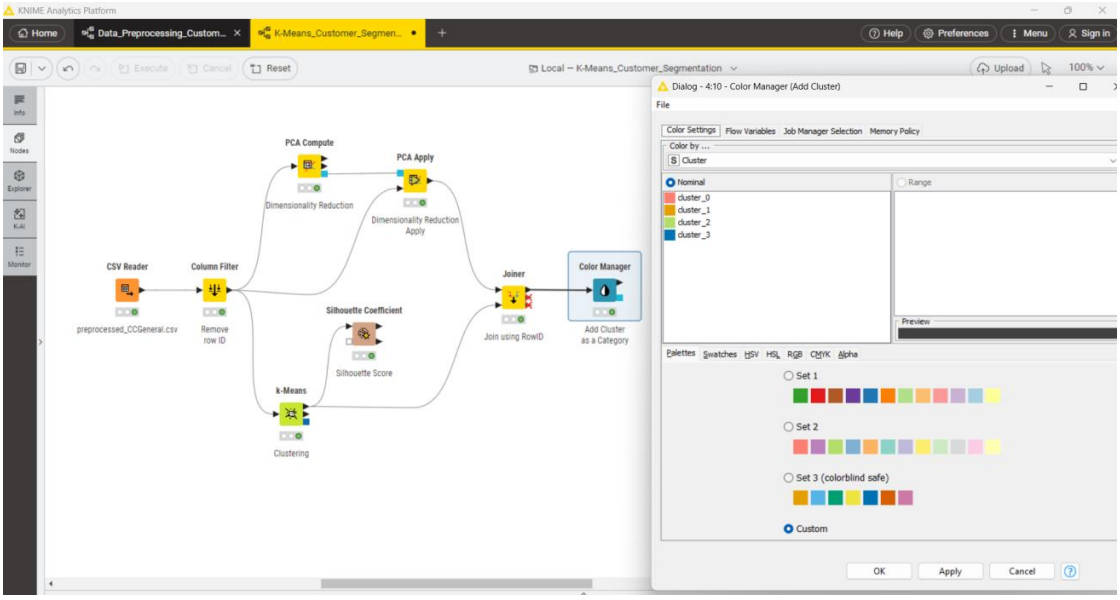


Figure 27: Color Manager Node and Configuration

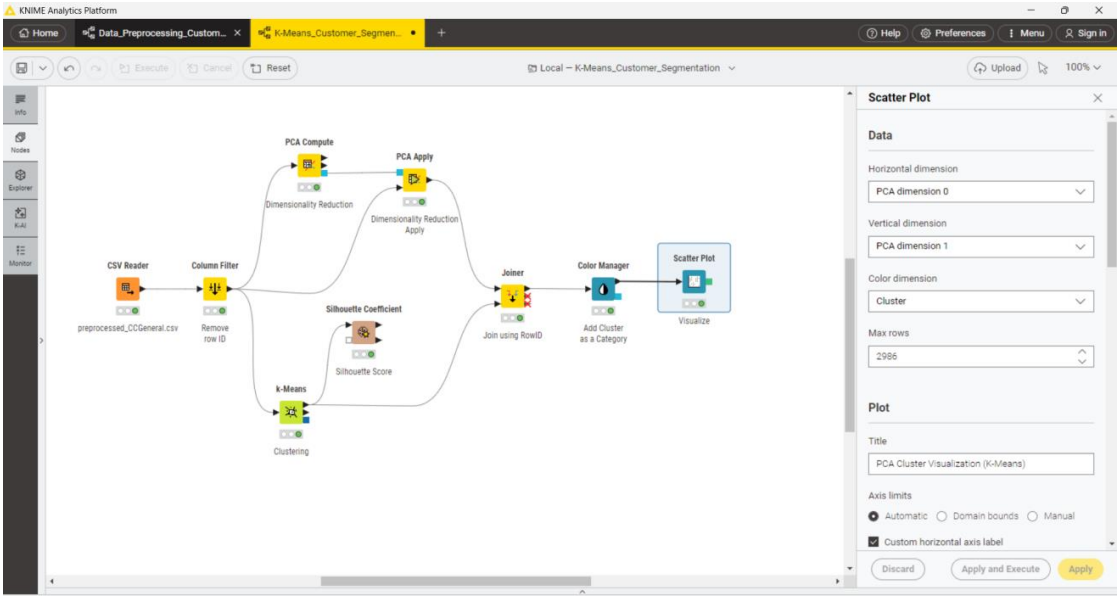


Figure 28: Scatter Plot Node and Configuration

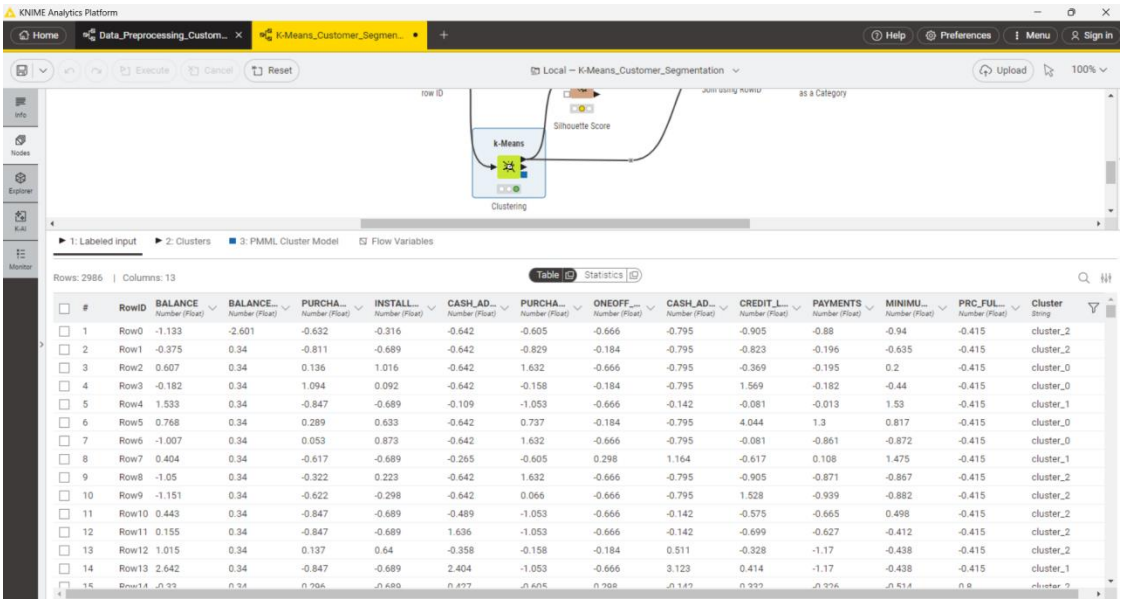
Clustering when k=4



Figure 29: Output diagram of Scatter Plot Node

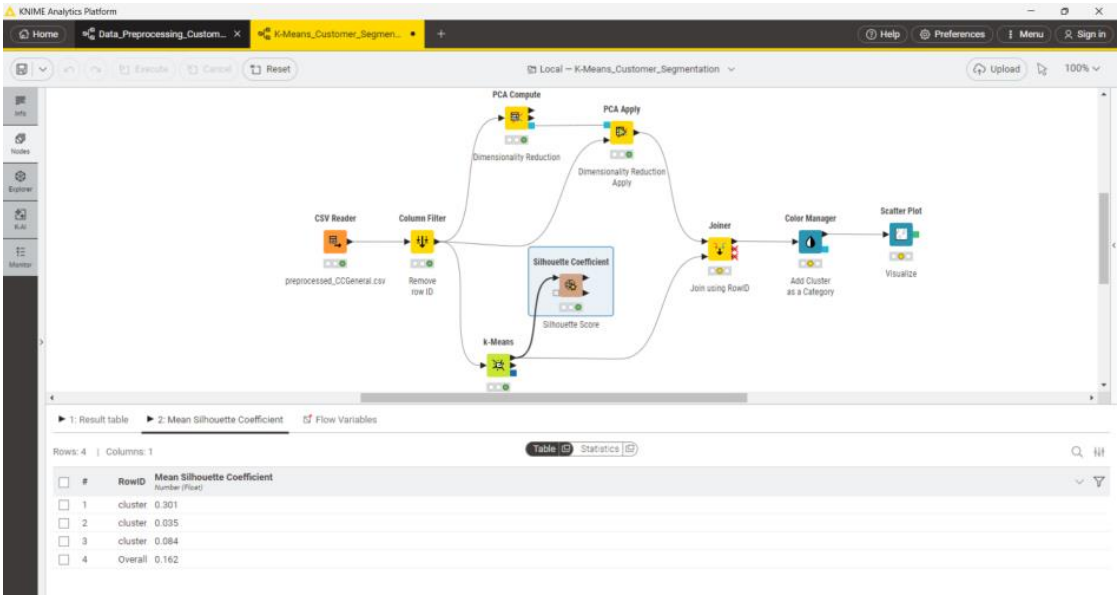
To visualize the resulting clusters, **Principal Component Analysis (PCA node)** was applied to reduce the multidimensional feature space into two principal components (PC1 and PC2). The transformed dataset was plotted using a **Scatter Plot node**, with each point colored by its assigned cluster label. This visualization provides a clear representation of cluster separation in reduced dimensions, allowing intuitive interpretation of customer groups.

Clustering when k=3



#	RowID	BALANCE	BALANCE...	PURCHA...	INSTALL...	CASH_AD...	PURCHA...	ONEOFF_...	CASH_AD...	CREDIT_L...	PAYMENTS	MINIMU...	PRC_FUL...	Cluster
1	Row0	-1.133	-2.601	-0.632	-0.316	-0.642	-0.605	-0.666	-0.795	-0.905	-0.88	-0.94	-0.415	cluster_2
2	Row1	-0.375	0.34	-0.811	-0.689	-0.642	-0.829	-0.184	-0.795	-0.823	-0.196	-0.635	-0.415	cluster_2
3	Row2	0.607	0.34	0.136	1.016	-0.642	1.632	-0.666	-0.795	-0.369	-0.195	0.2	-0.415	cluster_0
4	Row3	-0.182	0.34	1.094	0.092	-0.642	-0.158	-0.184	-0.795	1.569	-0.182	-0.44	-0.415	cluster_0
5	Row4	1.533	0.34	-0.847	-0.689	-0.109	-1.053	-0.666	-0.142	-0.081	-0.013	1.53	-0.415	cluster_1
6	Row5	0.768	0.34	0.289	0.633	-0.642	0.737	-0.184	-0.795	4.044	1.3	0.817	-0.415	cluster_0
7	Row6	-1.007	0.34	0.053	0.873	-0.642	1.632	-0.666	-0.795	-0.081	-0.861	-0.872	-0.415	cluster_0
8	Row7	0.404	0.34	-0.617	-0.689	-0.265	-0.605	0.298	1.164	-0.617	0.108	1.475	-0.415	cluster_1
9	Row8	-1.05	0.34	-0.322	0.223	-0.642	1.632	-0.666	-0.795	-0.905	-0.871	-0.867	-0.415	cluster_2
10	Row9	-1.151	0.34	-0.622	-0.298	-0.642	0.066	-0.666	-0.795	1.528	-0.939	-0.882	-0.415	cluster_2
11	Row10	0.443	0.34	-0.847	-0.689	-0.489	-1.053	-0.666	-0.142	-0.575	-0.665	0.498	-0.415	cluster_2
12	Row11	0.155	0.34	-0.847	-0.689	1.636	-1.053	-0.666	-0.142	-0.699	-0.627	-0.412	-0.415	cluster_2
13	Row12	1.015	0.34	0.137	0.64	-0.358	-0.158	-0.184	0.511	-0.328	-1.17	-0.438	-0.415	cluster_2
14	Row13	2.642	0.34	-0.847	-0.689	2.404	-1.053	-0.666	3.123	0.414	-1.17	-0.438	-0.415	cluster_1

Figure 30: Output Table of K-Means Node



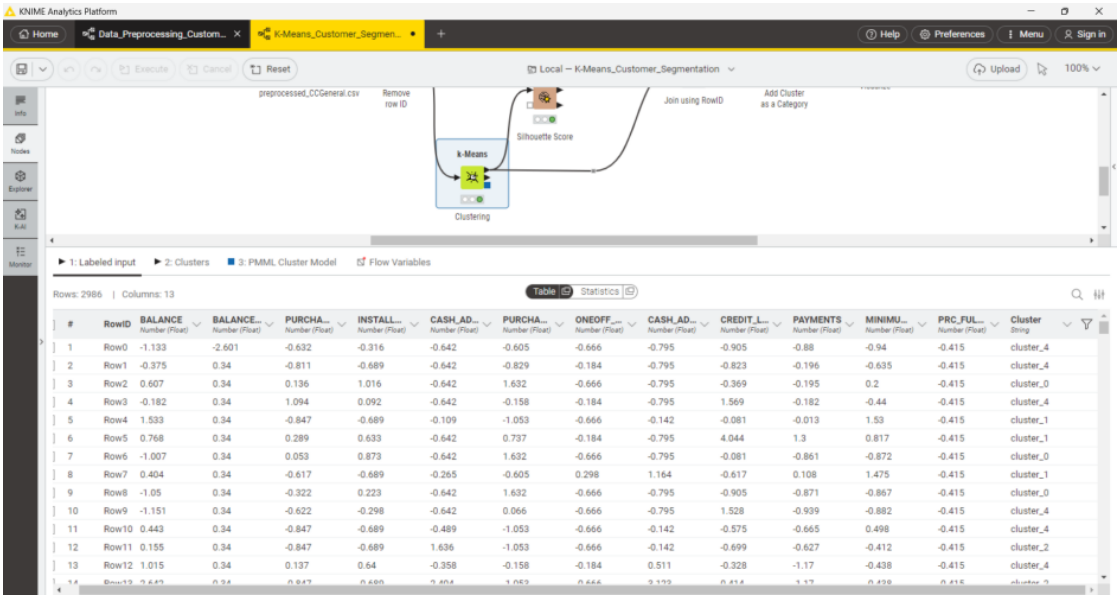
#	RowID	Mean Silhouette Coefficient
1	cluster	0.301
2	cluster	0.035
3	cluster	0.084
4	Overall	0.162

Figure 31: Output of Silhouette Coefficient Node - Mean Silhouette Coefficient



Figure 32: Output diagram of Scatter Plot Node

Clustering when k=5



#	RowID	BALANCE	BALANCE...	PURCHA...	INSTALL...	CASH_AD...	PURCHA...	ONEOFF_...	CASH_AD...	CREDIT_L...	PAYMENTS	MINIMU...	PRC_FUL...	Cluster
1	Row0	-1.133	-2.601	-0.632	-0.316	-0.642	-0.605	-0.666	-0.795	-0.905	-0.88	-0.94	-0.415	cluster_4
2	Row1	-0.375	0.34	-0.811	-0.689	-0.642	-0.829	-0.184	-0.795	-0.823	-0.196	-0.635	-0.415	cluster_4
3	Row2	0.607	0.34	0.136	1.016	-0.642	1.632	-0.666	-0.795	-0.369	-0.195	0.2	-0.415	cluster_0
4	Row3	-0.182	0.34	1.094	0.092	-0.642	-0.158	-0.184	-0.795	1.569	-0.182	-0.44	-0.415	cluster_4
5	Row4	1.533	0.34	-0.847	-0.689	-0.109	-1.053	-0.666	-0.142	-0.081	-0.013	1.53	-0.415	cluster_1
6	Row5	0.768	0.34	0.289	0.633	-0.642	0.737	-0.184	-0.795	4.044	1.3	0.817	-0.415	cluster_1
7	Row6	-1.007	0.34	0.053	0.873	-0.642	1.632	-0.666	-0.795	-0.081	-0.861	-0.872	-0.415	cluster_0
8	Row7	0.404	0.34	-0.617	-0.689	-0.265	-0.605	0.298	1.164	-0.617	0.108	1.475	-0.415	cluster_1
9	Row8	-1.05	0.34	-0.322	0.223	-0.642	1.632	-0.666	-0.795	-0.905	-0.871	-0.867	-0.415	cluster_0
10	Row9	-1.151	0.34	-0.622	-0.298	-0.642	0.066	-0.666	-0.795	1.528	-0.939	-0.882	-0.415	cluster_4
11	Row10	0.443	0.34	-0.847	-0.689	-0.489	-1.053	-0.666	-0.142	-0.575	-0.665	0.498	-0.415	cluster_4
12	Row11	0.155	0.34	-0.847	-0.689	1.636	-1.053	-0.666	-0.142	-0.699	-0.627	-0.412	-0.415	cluster_2
13	Row12	1.015	0.34	0.137	0.64	-0.358	-0.158	-0.184	0.511	-0.328	-1.17	-0.438	-0.415	cluster_4

Figure 33: Output Table of K-Means Node

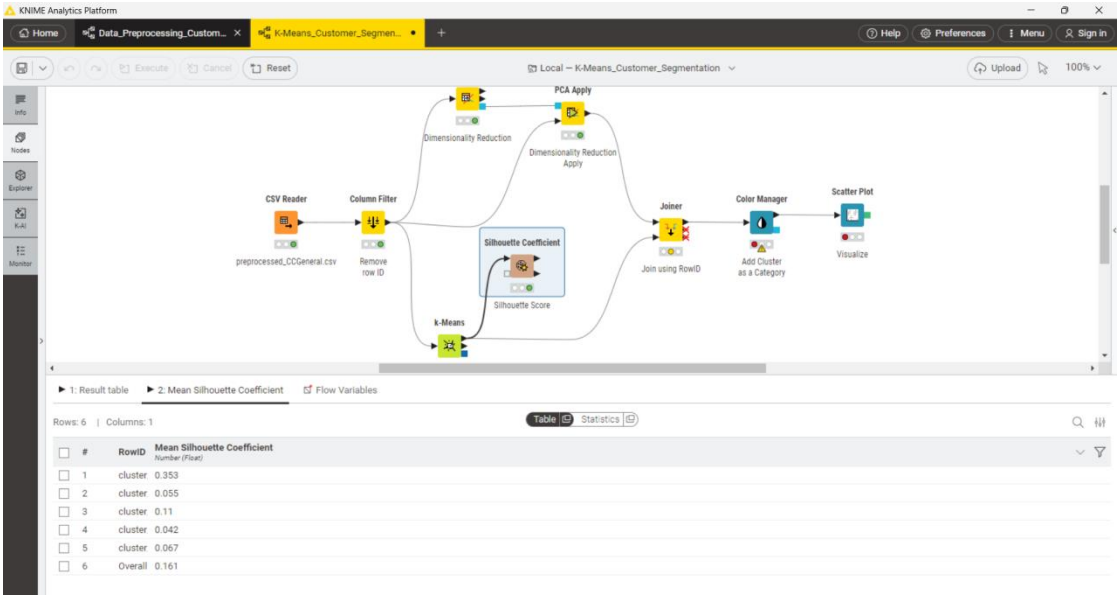


Figure 34: Output of Silhouette Coefficient Node - Mean Silhouette Coefficient

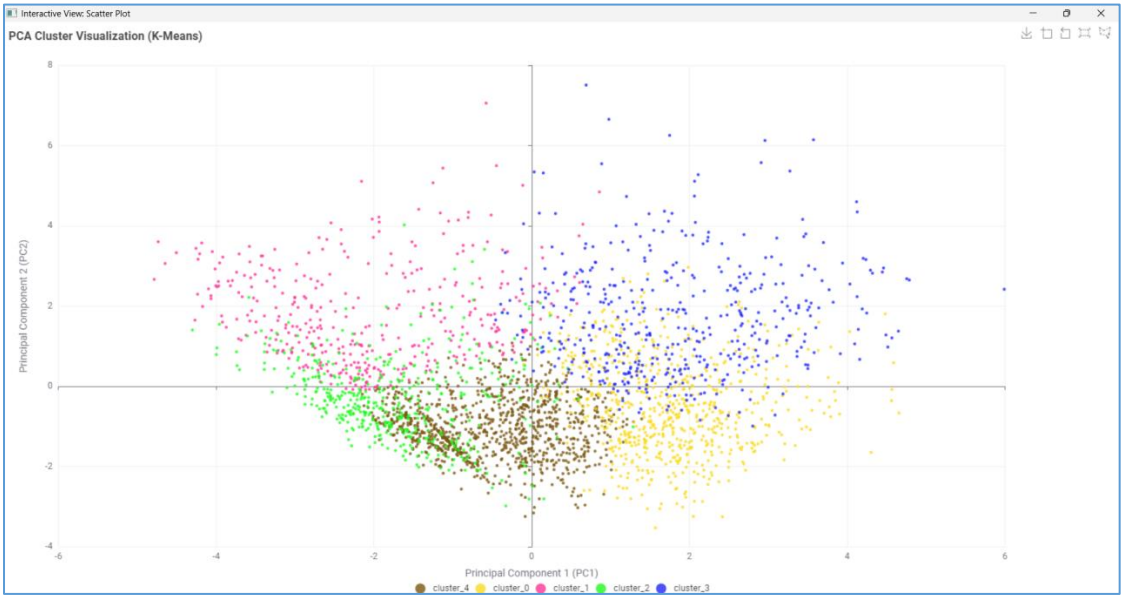


Figure 35: Output diagram of Scatter Plot Node

Summary of K-Means Clustering

To determine the optimal number of clusters, the K-Means model was executed with k = 3, 4, and 5, and the Silhouette Coefficient was used to evaluate the compactness

and separation of the clusters. Among the tested values, k = 3 achieved the highest overall silhouette score (0.1617) compared to k = 5 (0.1605) and k = 4 (0.1388). Although the silhouette values are relatively low—typical for high-dimensional financial datasets—the result indicates slightly better-defined cluster boundaries when the data is partitioned into three groups. Therefore, k = 3 was selected as the most appropriate number of clusters, offering the best balance between cohesion and separation for customer segmentation in this dataset.

(b) Hierarchical Clustering

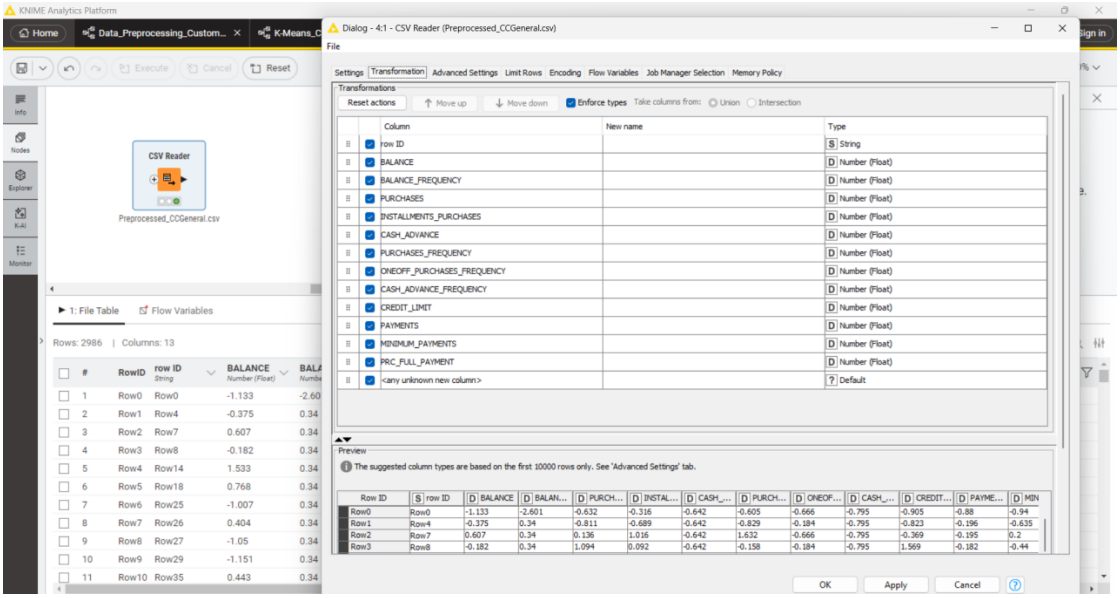


Figure 36: CSV Reader Node, Configuration, and Output Table

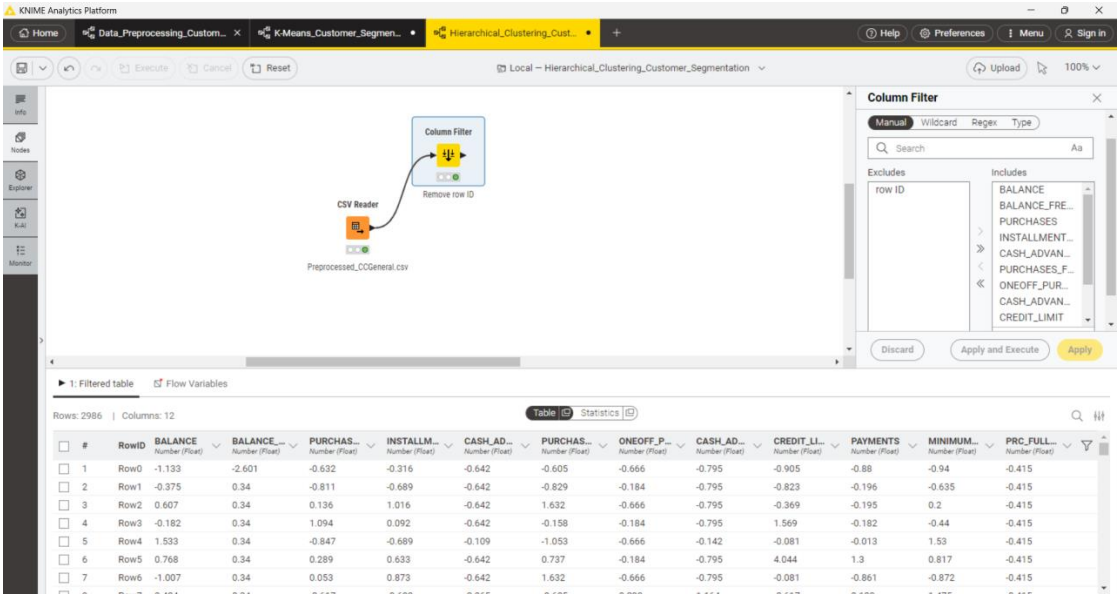


Figure 37: Column Filter Node, Configuration, and Output

The *row ID* column was removed because it is only an index and contains no meaningful information for clustering. Keeping it would add noise and distort distance calculations.

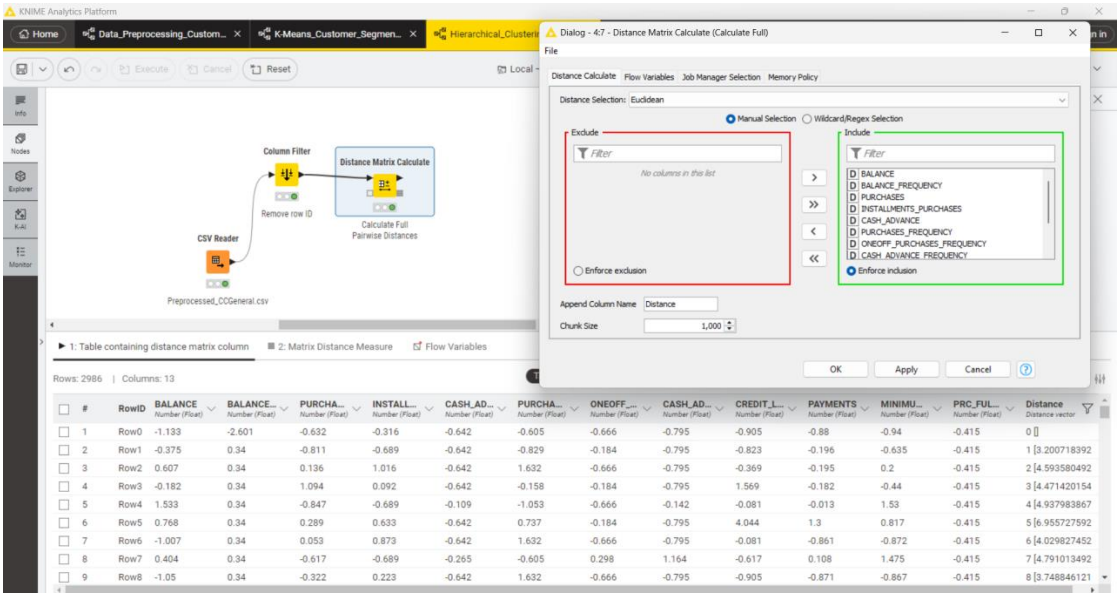


Figure 38: Distance Matrix Calculate Node, Configuration, and Output Table

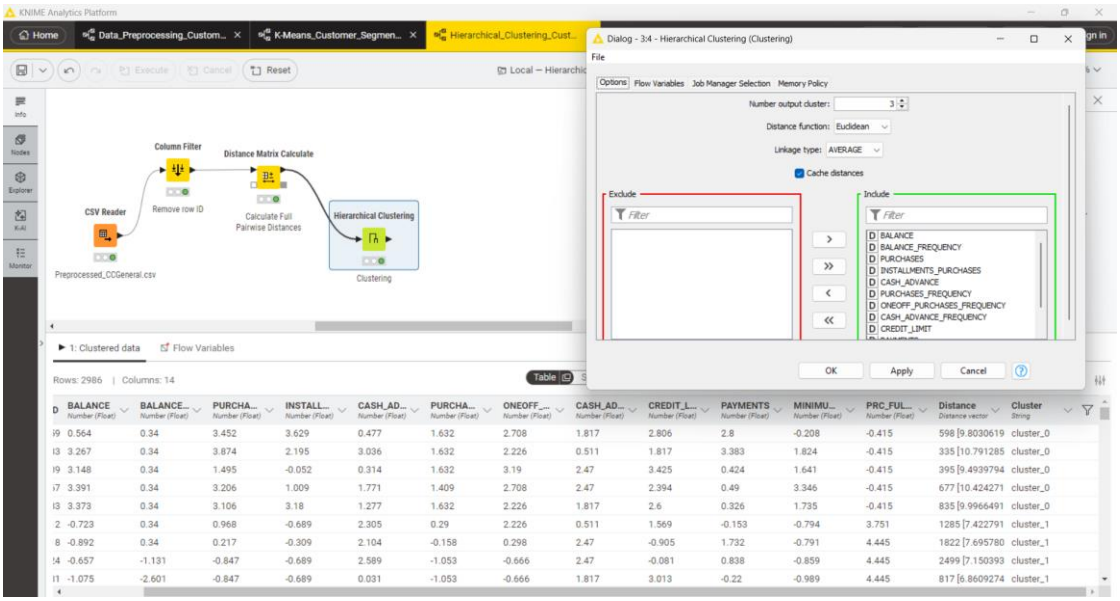


Figure 39: Hierarchical Clustering Node, Configuration, and Output Table

Hierarchical Clustering was performed using average linkage in the **Hierarchical Clustering node**, which provides a balanced approach between the overly strict complete linkage and the noise-sensitive single linkage. This method computes the

average distance between points in different clusters, producing stable and meaningful cluster structures for normalized numerical data.

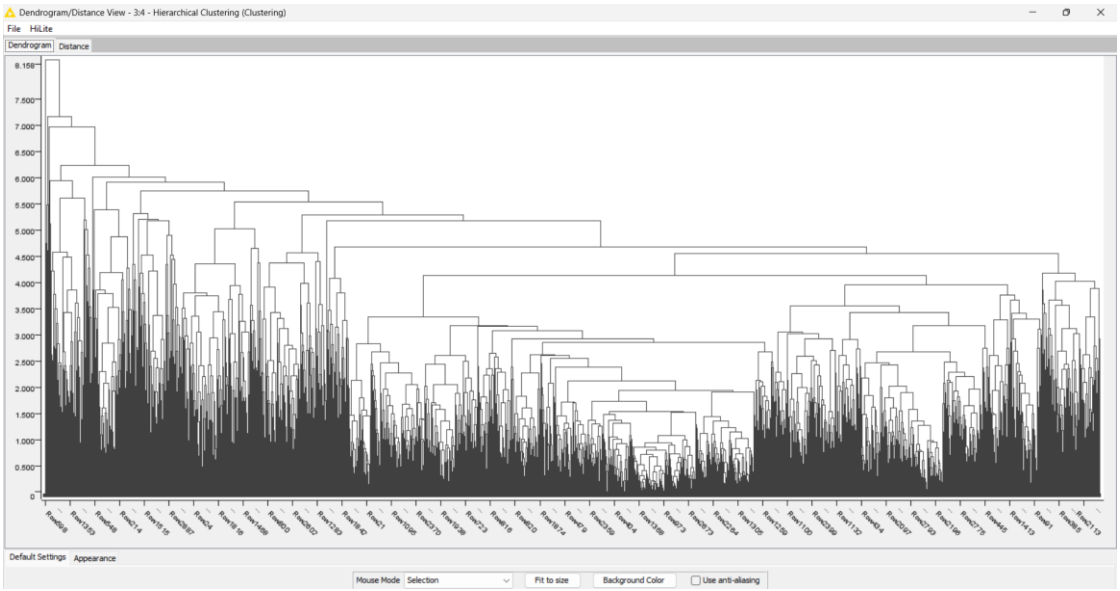


Figure 40: Dendrogram/Distance View from Hierarchical Clustering Node

The dendrogram shows a large vertical distance jump around height 4.
Cutting the dendrogram at height ≈ 4.0 naturally separates the dataset into three well-defined clusters with good separation.

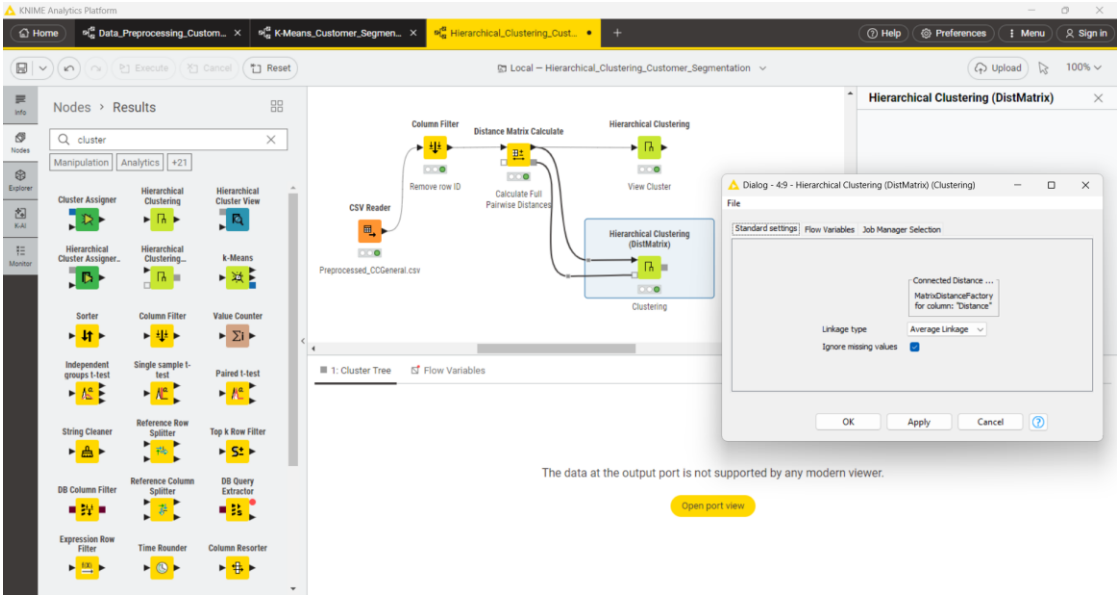


Figure 41: Hierarchical Clustering (DistMatrix) Node and Configuration

Although Hierarchical Clustering was performed using the **Hierarchical Clustering node** its output is not compatible with the Hierarchical Cluster Assigner (which is to be used in next step), as it does not provide a cluster-tree output. Therefore, the **Hierarchical Clustering (DistMatrix) node** was additionally used to generate the required cluster tree for assigning final cluster labels.

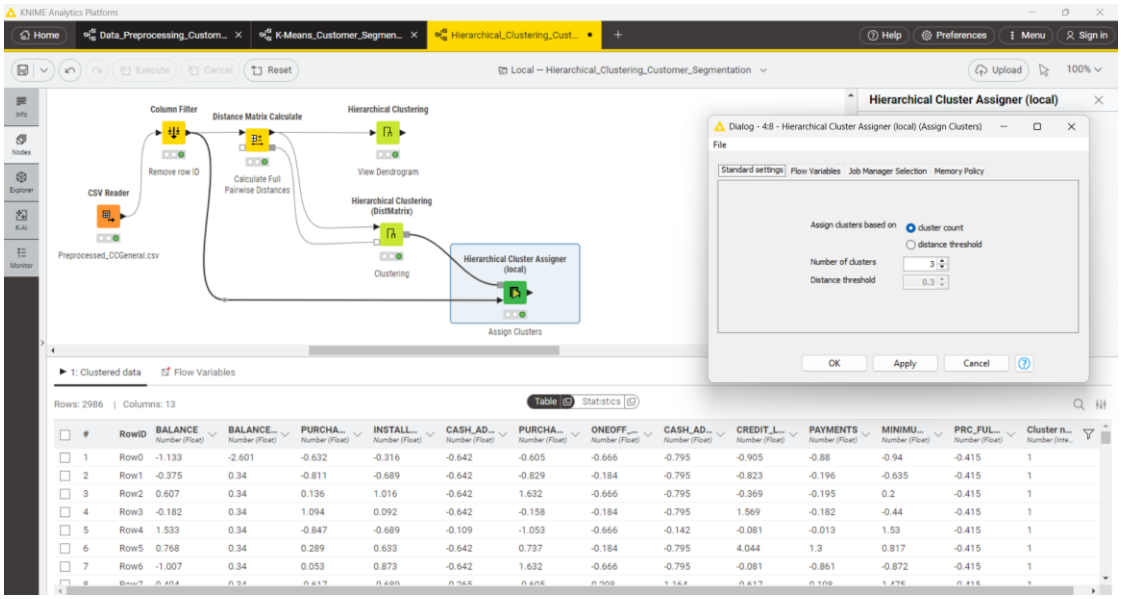


Figure 42: Hierarchical Clustering Assigner Node, Configuration, and Output

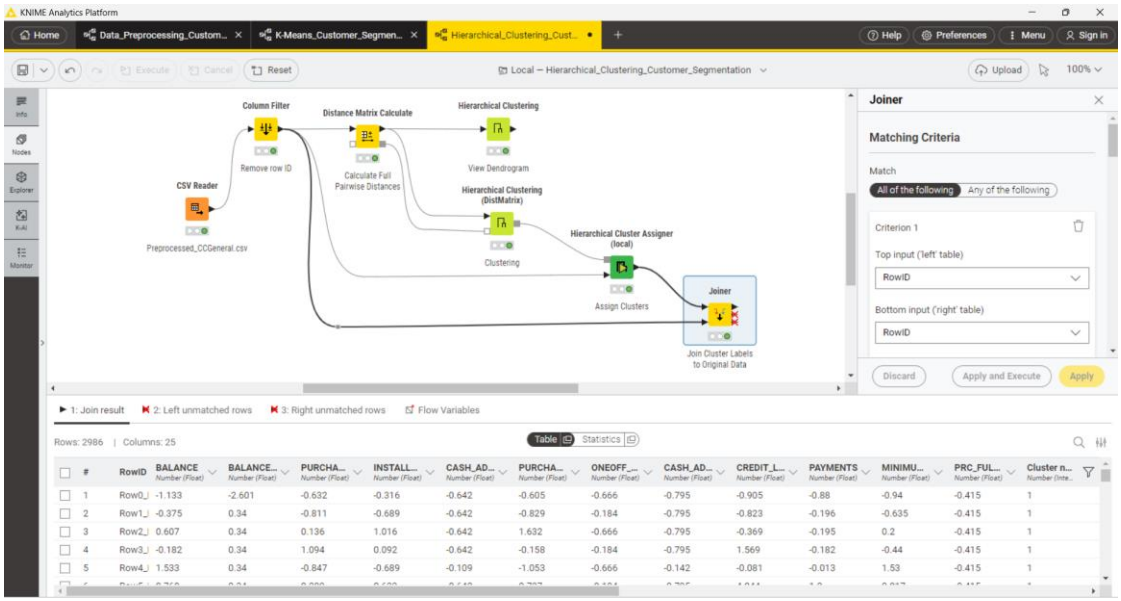


Figure 43: Joiner Node, Configuration, and Output

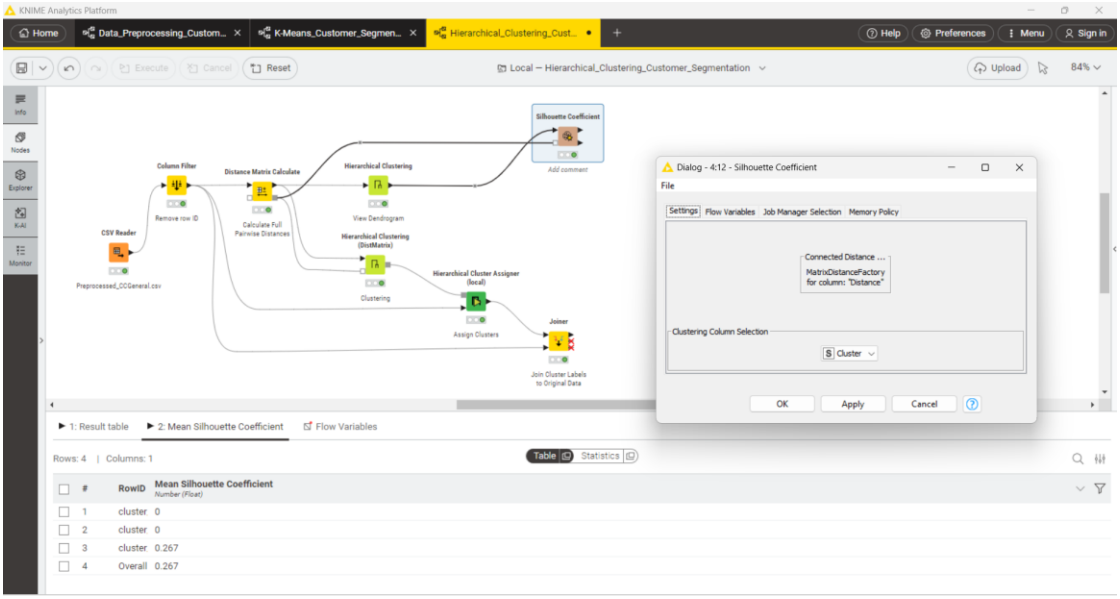


Figure 44: Silhouette Coefficient Node, Configuration, and Output Table - Mean Silhouette Coefficient

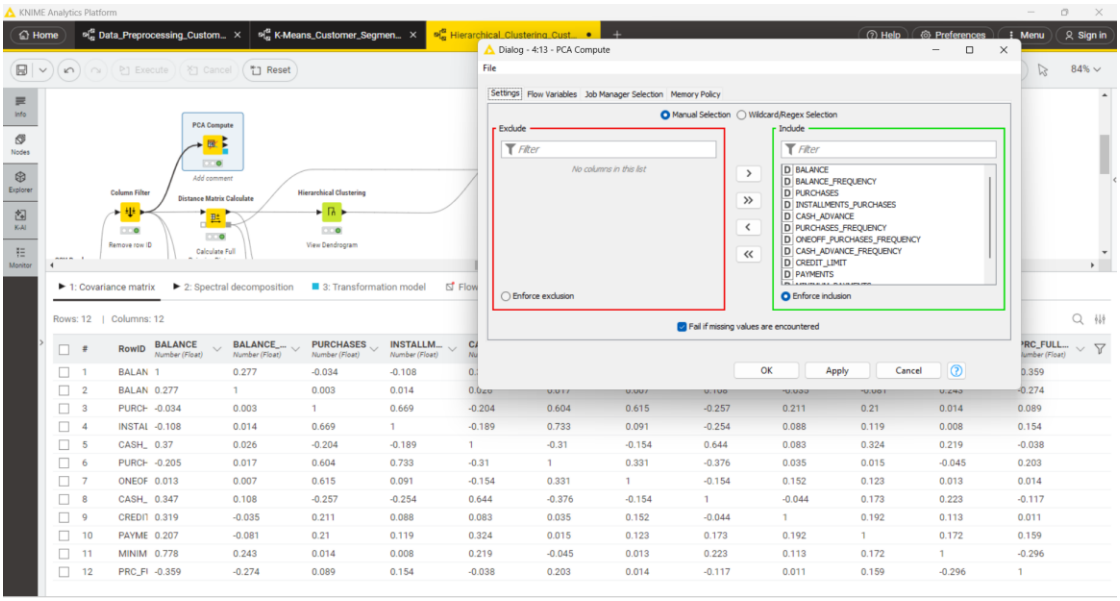


Figure 45: PCA Compute Node and Configuration

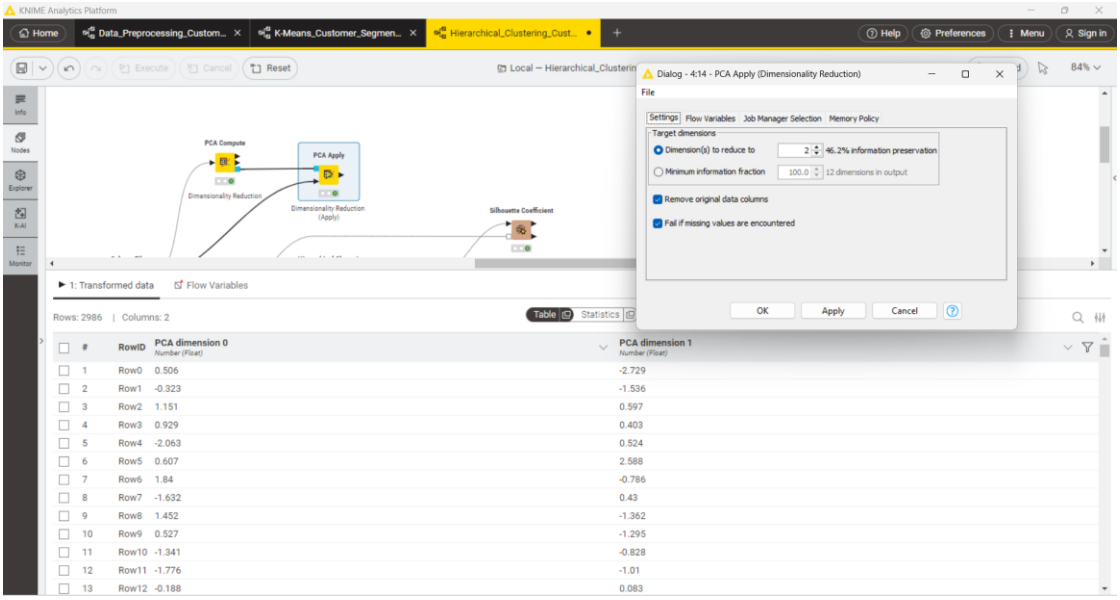


Figure 46: PCA Apply Node and Configuration

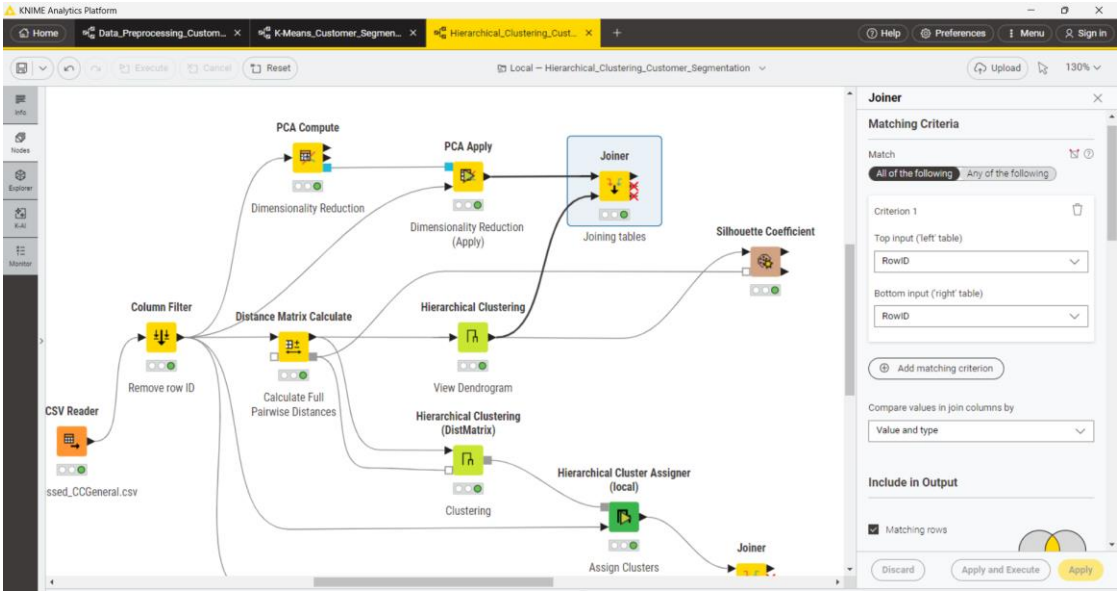


Figure 47: Joiner Node and Configuration

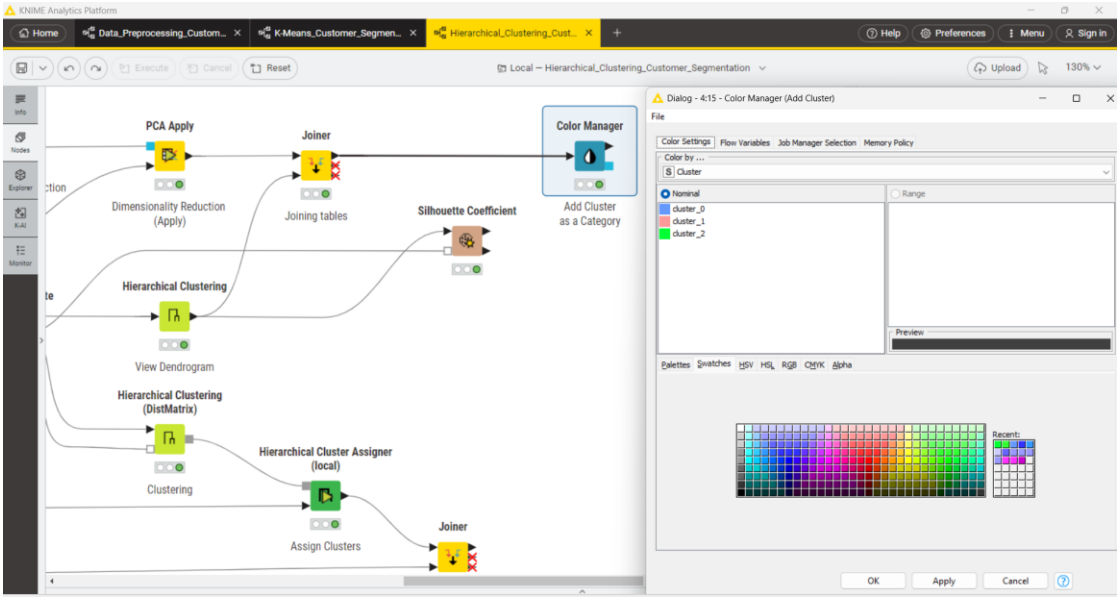


Figure 48: Color Manager Node and Configuration

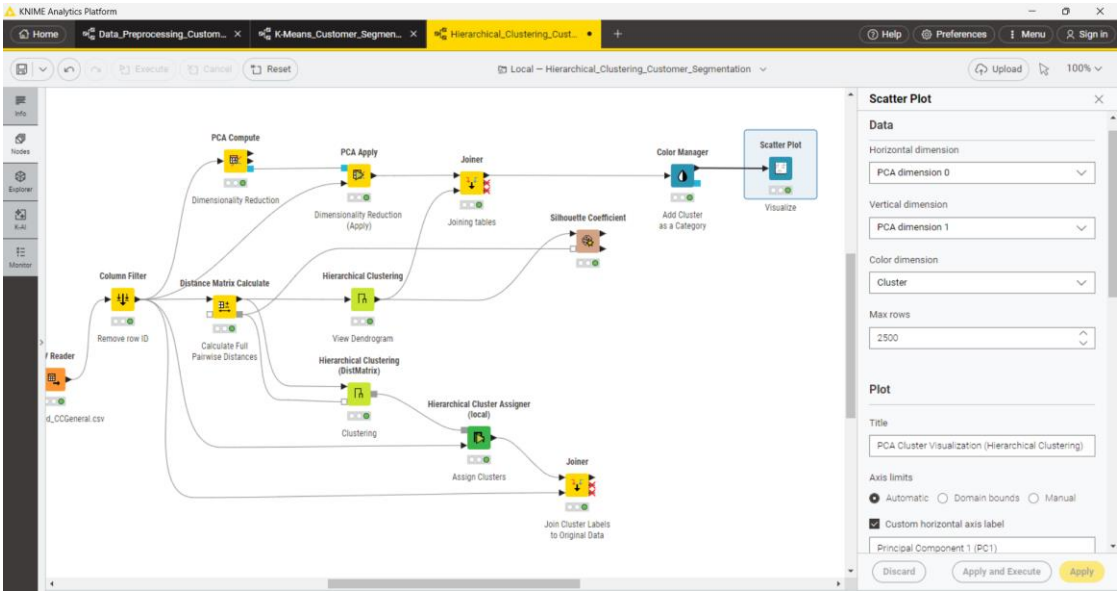


Figure 49: Scatter Plot Node and Configuration

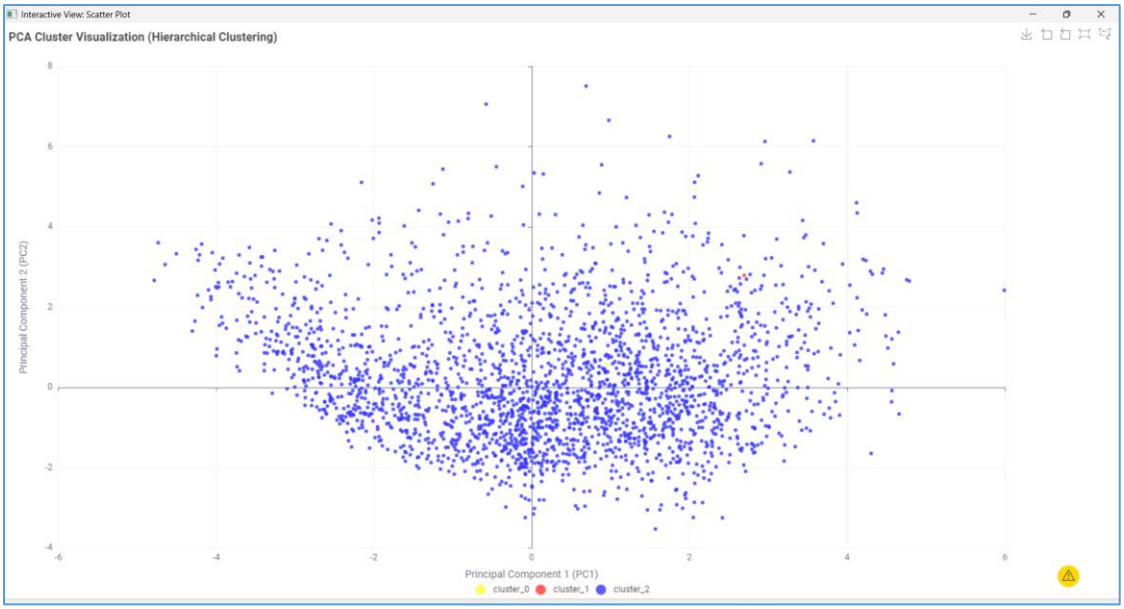


Figure 50: Output diagram of Scatter Plot Node (Hierarchical Clustering)

(c) DBSCAN

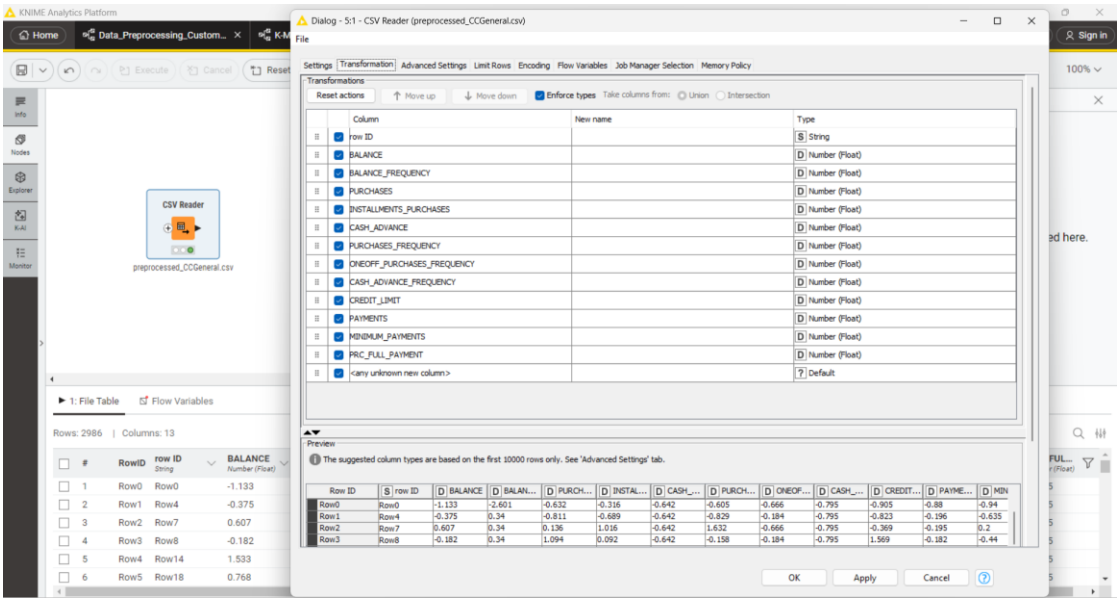


Figure 51: CSV Reader Node, Configuration, and Output Table

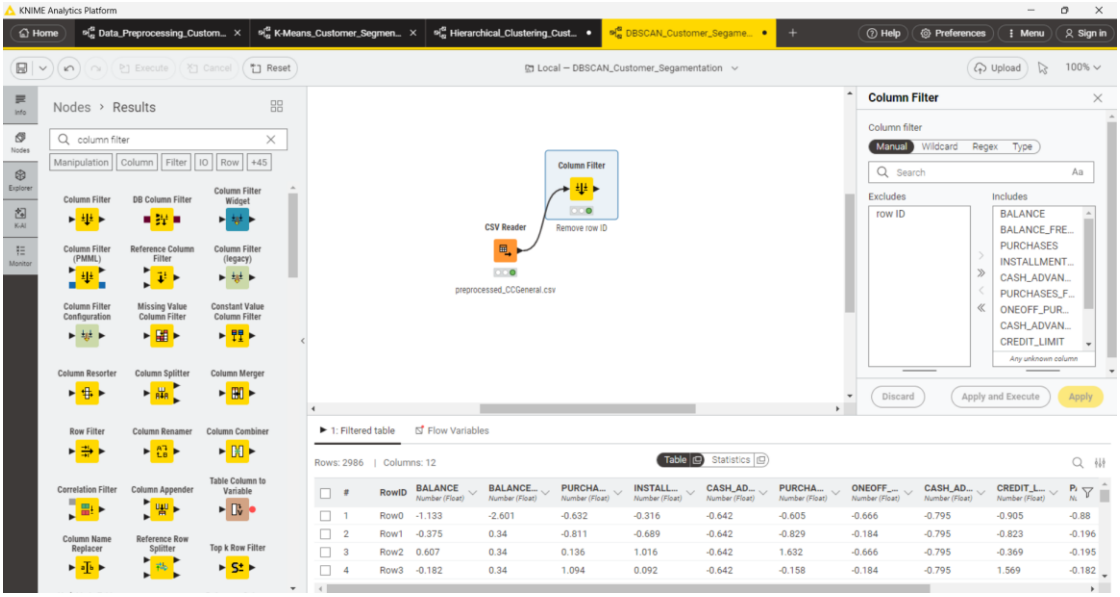


Figure 52: Column Filter Node, Configuration, and Output

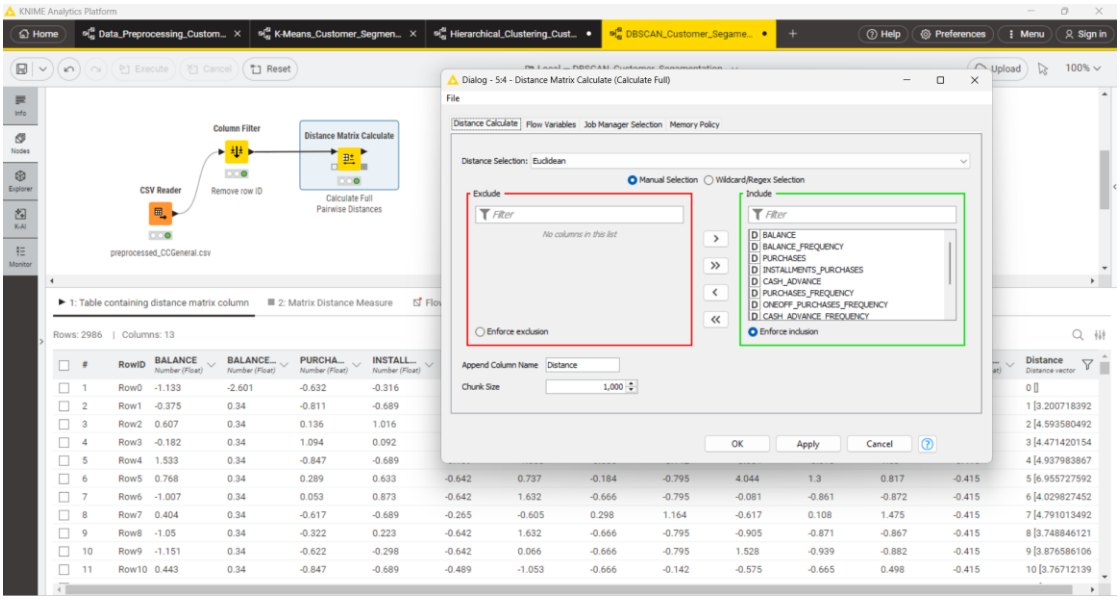


Figure 53: Distance Matrix Calculate, Configuration, and Output

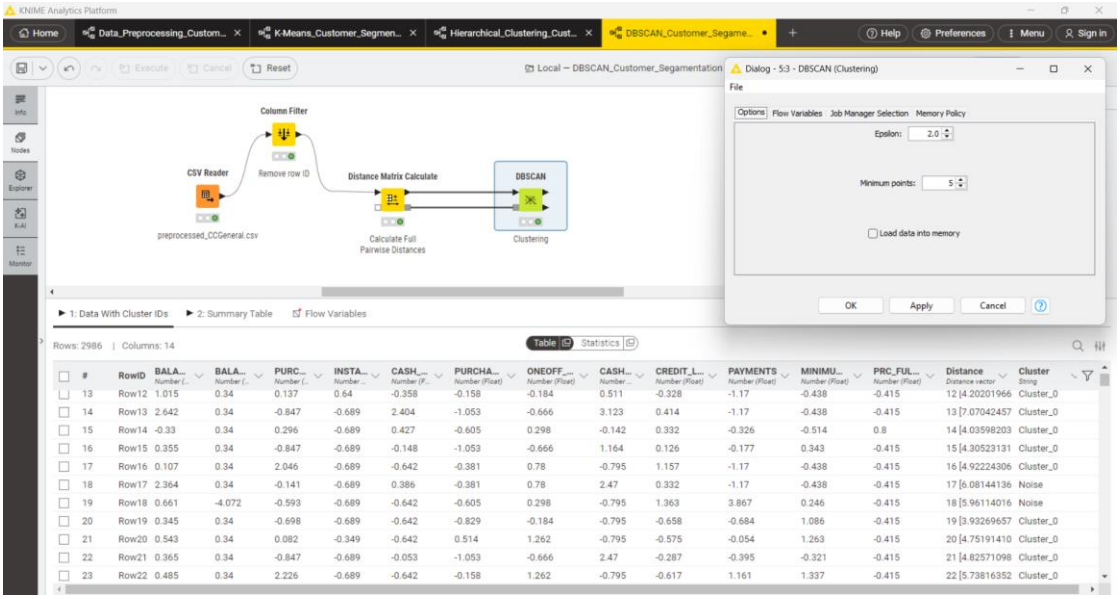


Figure 54: DBSCAN Node, Configuration, and Output

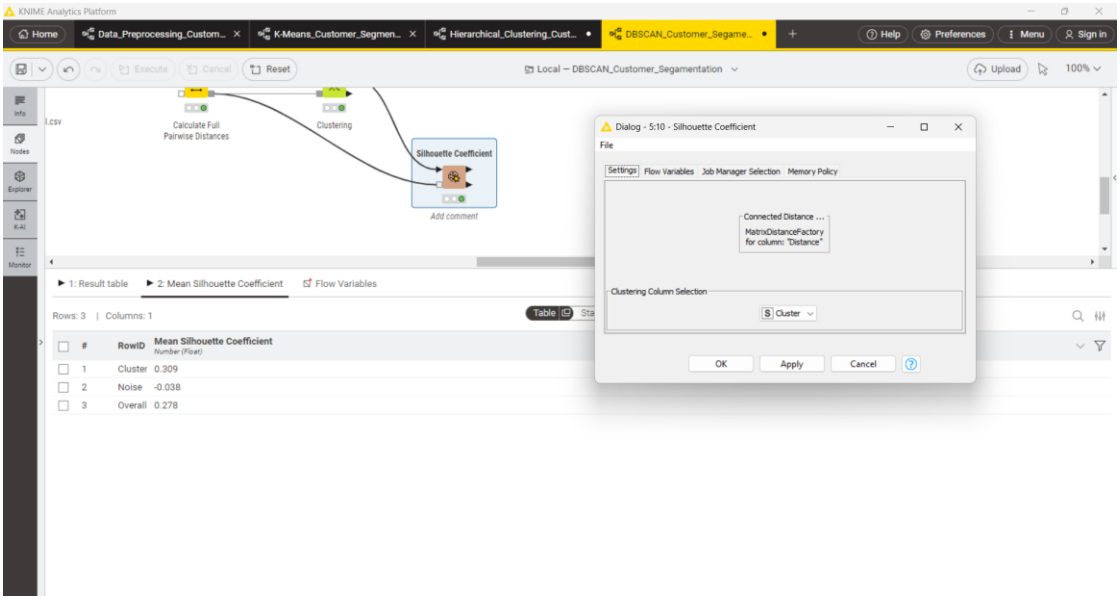


Figure 55: Silhouette Coefficient Node, Configuration, and Output

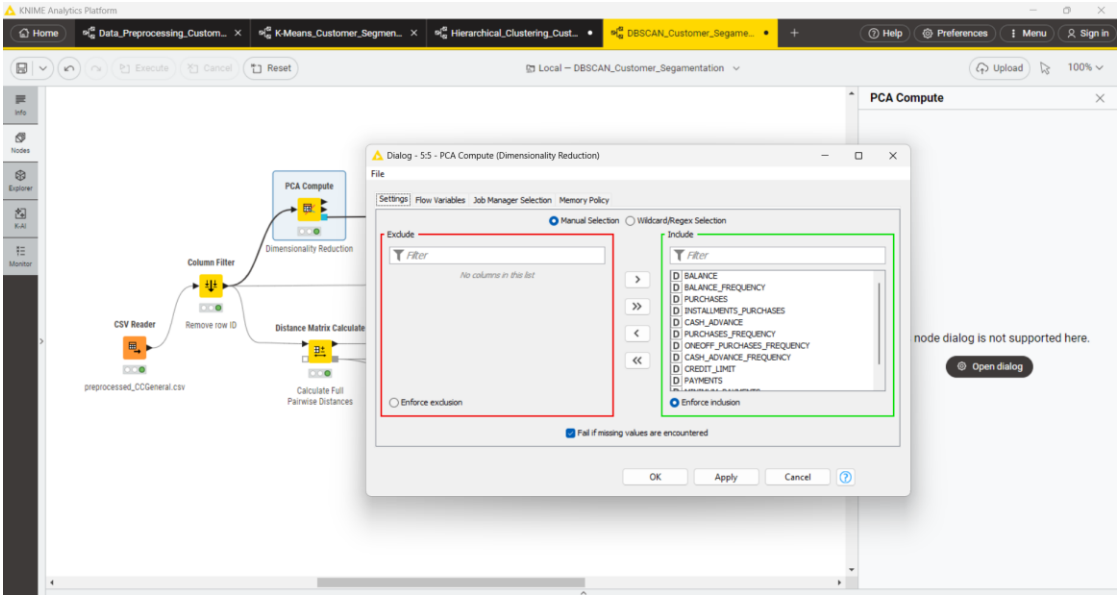


Figure 56: PCA Compute Node and Configuration

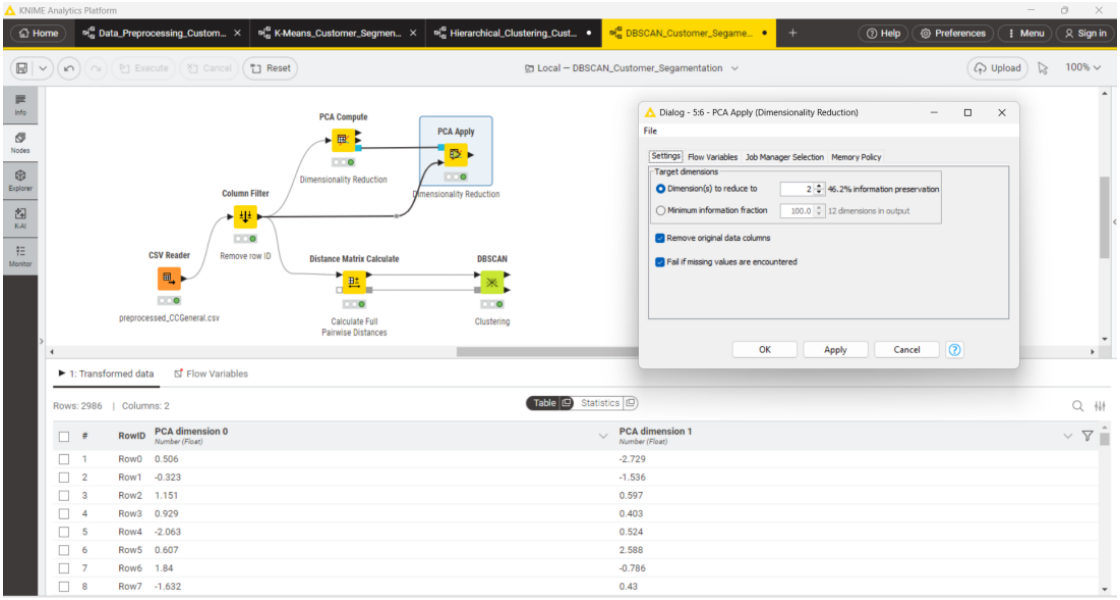


Figure 57: PCA Apply Node and Configuration

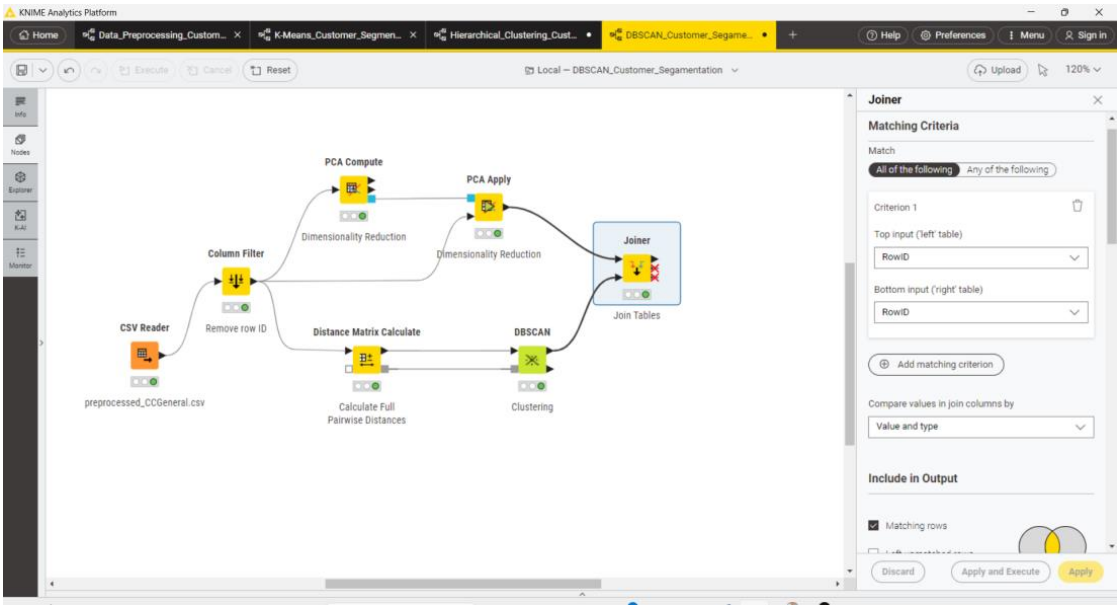


Figure 58: Joiner Node and Configuration

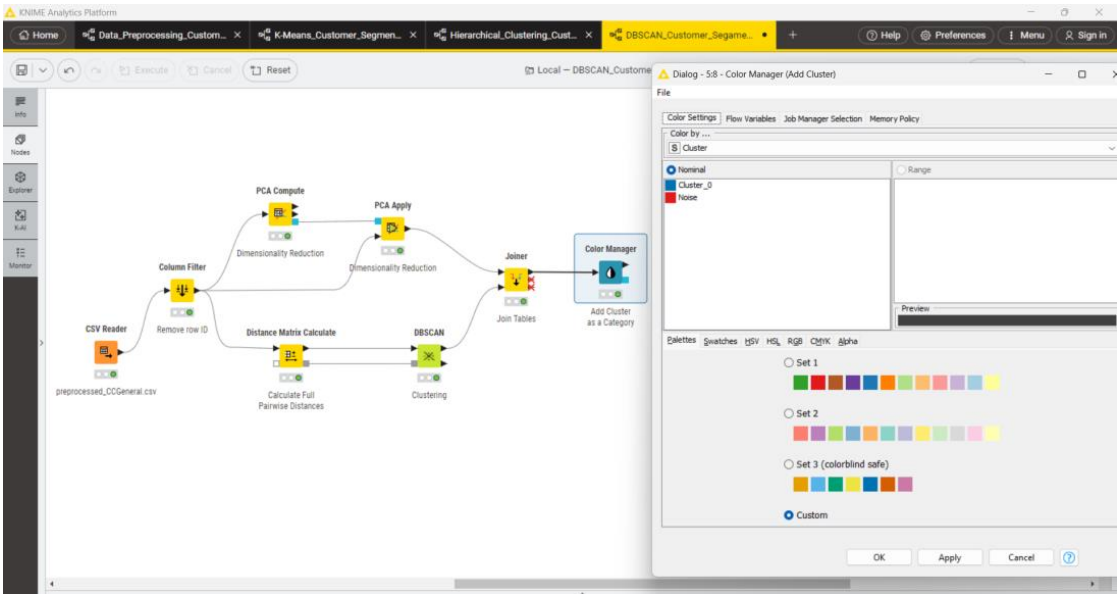


Figure 59: Color Manager Node and Configuration

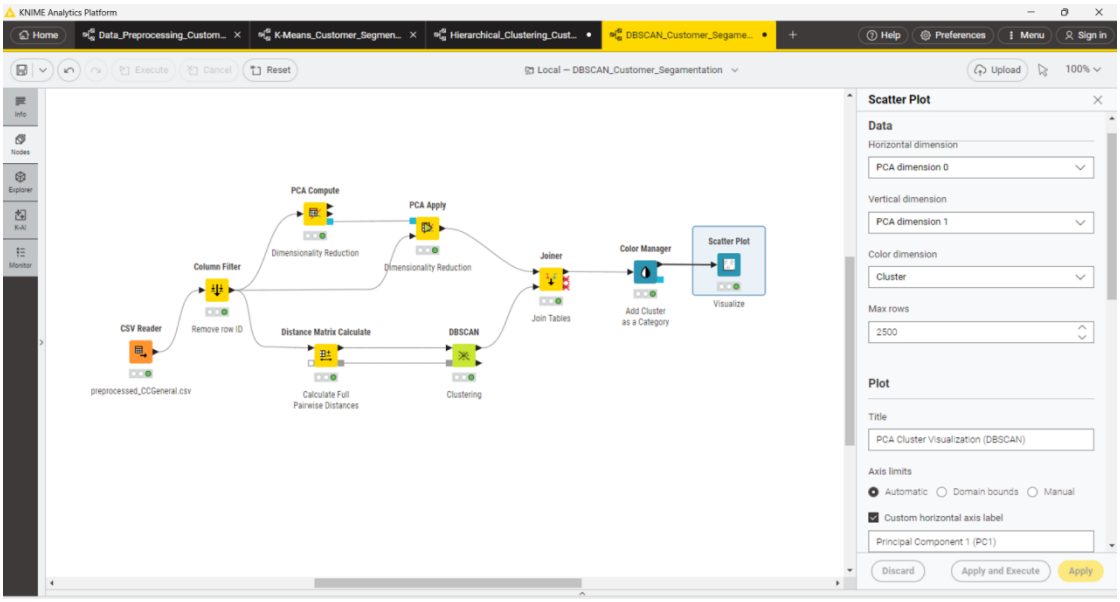


Figure 60: Scatter Plot Node and Configuration



Figure 61: Output diagram of Scatter Plot Node